

Rapport de projet multidisciplinaire

Modélisation du quadrupôle d'Okayama par BEM

(Boundary Element Method)

Zoé Nonon – Louise Lammertyn
4^{ème} année Génie Physique – INSA Toulouse
Année universitaire 2025-2026

Tuteurs :

Florent Houdellier
Lucile Berdoux Durieux
Jean-Baptiste Mellier
Pier-Francesco Fazzini

Tescan

Rapport de projet multidisciplinaire

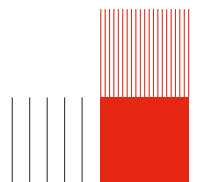
Modélisation du quadrupôle d'Okayama par BEM (*Boundary Element Method*)

Zoé Nonon – Louise Lammertyn
4^{ème} année Génie Physique – INSA Toulouse
Année universitaire 2025-2026

Tuteurs :

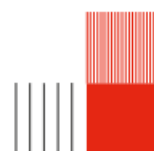
Florent Houdellier
Lucile Berdoux Durieux
Jean-Baptiste Mellier
Pier-Francesco Fazzini

Tescan



Remerciements

Ce projet multidisciplinaire n'aurait pu voir le jour sans l'aide de...



Abstract

Ce document est le rapport final du projet multidisciplinaire de 4ème année en Génie Physique de l'INSA Toulouse. Il a pour but d'expliquer en détail la démarche que nous avons suivie afin de mener à bien ce projet. Ce document retrace donc le contexte, l'organisation mais aussi toutes les équations que nous avons dû étudier.

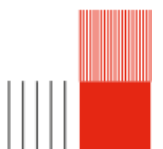
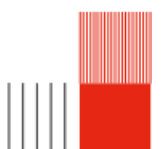


Table des matières

Remerciements	3
Remerciements	4
1 Introduction	6
1.1 Contexte	6
1.2 Fondement théorique	6
1.2.1 L'optique des particules chargés	6
1.2.2 Principe de correction d'aberration	7
1.2.3 Le quadrupole d'okayama	7
1.3 Objectifs du projet	8
1.4 Livrables	8
2 Travail Réalisé	9
2.1 Prérequis mathématique	9
2.1.1 Principe de moindre action	9
2.1.2 Equations d'Euler-Lagrange	9
2.1.3 Décomposition du potentiel	9
2.2 Décomposition des différentes contributions	11
2.3 Reconstruction de la décomposition du potentiel	11
2.3.1 Extraction du potentiel	11
2.3.2 Composantes multipolaires	11
2.4 Trajectoire	15
2.4.1 Principe de fermat	15
2.4.2 Cas $\nu = 2$, quadrupole	15
2.4.3 Equation d'euler lagrange	16
2.4.4 Résolution equa diff(cas sans polariser les apertures)	16
2.4.5 Vérification des résultats	16
2.5 Document Okayama	18
3 Notre production	19
3.1 Structure globale du code	19
3.2 Codes utilisateur	19
3.2.1 Extraction du potentiel	19
3.2.2 Exploitation des données	20
3.3 Programme principal	20
3.3.1 Data	20
3.3.2 Geometry	21
3.3.3 Field calculation	25
3.3.4 Extraction data	28
3.3.5 Multipolar decomposition	30
3.3.6 Fit functions	31
3.3.7 Graphs	33
3.3.8 Paraxial	35



Chapitre 1

Introduction

1.1 Contexte

Dans le cadre du projet multidisciplinaire de 4ème du Génie physique de l'INSA Toulouse, nous devons modéliser les lentilles électrostatiques quadrupolaires d'un FIB (Focused Ion Beam) en utilisant une méthode mathématique appelée la méthode des éléments de frontière. Pour ce faire, nous devons écrire un code Python en utilisant notamment la bibliothèque BEMPP. Afin de pouvoir réaliser cette entreprise, il est nécessaire de comprendre l'optique des particules chargées dans le contexte de lentilles quadrupolaires.

1.2 Fondement théorique

1.2.1 L'optique des particules chargés

L'optique est une science divisée en deux domaines majeurs. L'optique photonique étudie le comportement de la lumière à travers des lentilles ou des miroirs. Cette technologie est utilisée dans les microscopes classiques, les télescopes ou les appareils photos. Cependant, la résolution de ces systèmes rencontre une limite physique : la diffraction.

En effet la résolution d'un système optique définit la capacité à distinguer deux points à distance proche. La résolution peut s'exprimer avec le critère critique de Rayleigh

$$d = \frac{1.22 \times \lambda}{n \sin(\alpha)}$$

avec :

- λ est la longueur d'onde de la particule.
- n est l'indice de réfraction du milieu.
- α est l'angle de demi-ouverture du faisceau.

Or la lumière possède une longueur d'onde entre 400 et 800 nm, paramètre limitant la résolution des systèmes optiques.

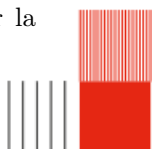
L'optique des particules chargées permet de s'affranchir de cette limite. Selon de Broglie, la longueur d'onde d'une particule chargée dépend de sa quantité de mouvement :

$$\lambda = \frac{h}{mv}$$

Pour des particules accélérées, la longueur d'onde devient extrêmement petite. Cette propriété permet de réduire les effets de la diffraction.

Applications et instrumentation

L'optique des particules chargées permet de maîtriser la trajectoire de faisceaux d'électrons ou d'ions. Des champs électrostatiques ou magnétiques agissent comme des lentilles. Cette discipline a permis de créer de nombreux instruments comme les SEM (Scanning electron microscope = microscope électronique à balayage) et les FIB (focused ion beam = faisceau d'ions focalisé). Ces outils sont utilisés dans de nombreuses disciplines comme la médecine ou la microélectronique. Plus spécifiquement, un FIB est composé de plusieurs étages dont notamment des lentilles. En effet, celles-ci permettent de guider la



trajectoire du faisceau ionique au sein de l'instrument et de corriger les différentes aberrations de celui-ci. Il est donc primordial de comprendre et maîtriser ces objets afin de rendre le FIB le plus performant possible.

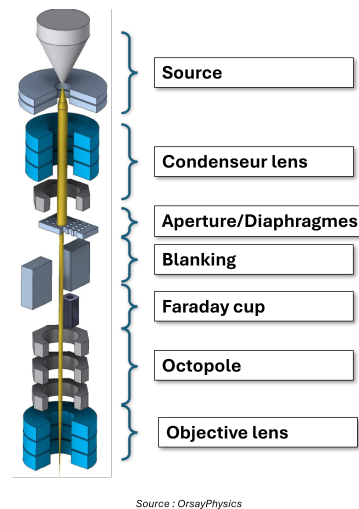


FIGURE 1.1 – Schéma d'un FIB

Comme évoqué précédemment, les lentilles permettent de guider la trajectoire du faisceau ionique. Il existe plusieurs types de lentilles : les lentilles électrostatiques et les lentilles magnétiques. Dans le cas que nous étudions, le FIB est composé de lentilles électrostatiques. Ces lentilles électrostatiques peuvent présenter différentes configurations. On parle en effet de lentilles rondes/monopolaires (un seul potentiel), de lentilles dipolaires (deux potentiels), de lentilles quadrupolaires (quatre potentiels), etc.

1.2.2 Principe de correction d'aberration

L'usage de particules chargées réduit l'impact de la diffraction. Cependant, la résolution reste limitée par des aberrations liées aux imperfections des champs. On distingue deux types principaux d'aberrations : les aberrations chromatiques, liées à la dispersion en énergie des particules, et les aberrations géométriques.

Parmi les aberrations géométriques, on trouve notamment l'aberration sphérique, le coma, l'astigmatisme ou la distorsion. Ce rapport se concentre sur l'aberration sphérique. Ce défaut traduit le fait que les particules s'éloignant de l'axe optique sont focalisées plus fortement que les particules centrales. Il en résulte un élargissement du point focal dans le plan image,

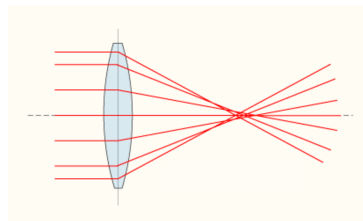
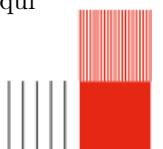


FIGURE 1.2 – aberration sphérique

La méthode classique pour corriger l'aberration sphérique utilise des correcteurs multipolaires. La superposition de champs quadrupolaires et octopolaires permet de compenser ce défaut. (mettre pourquoi) Cependant, l'alignement de ces nombreux étages représente un défi technique majeur.

1.2.3 Le quadrupole d'okayama

L'équipe de S. Okayama a travaillé sur un nouveau moyen de correction. Ce système évite l'utilisation d'électrodes octopolaires dédiées. En combinant un champ rond créé par une ouverture et un champ quadrupolaire, un champ octopolaire apparaît naturellement. Ce champ est dit « auto-aligné », ce qui simplifie grandement la conception et le réglage du correcteur.



1.3 Objectifs du projet

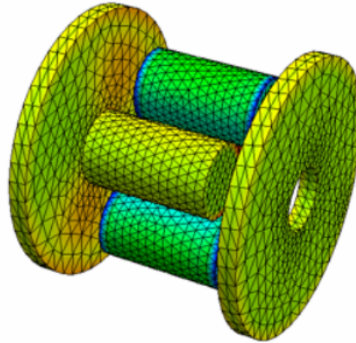


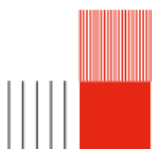
FIGURE 1.3 – Modélisation d’une lentille quadrupolaire électrostatique

Cependant, la fabrication de ces instruments peut parfois être longue et coûteuse. Pour pallier ce problème, l’utilisation de modèles s’avère utile. C’est dans ce cadre que notre projet multidisciplinaire s’inscrit et est intitulé : “Modélisation de lentilles multipolaires électrostatiques par Boundary Element Method”. Comme évoqué précédemment, les lentilles permettent de guider la trajectoire du faisceau ionique. Il existe plusieurs types de lentilles : les lentilles électrostatiques et les lentilles magnétiques. Dans le cas que nous étudions, le FIB est composé de lentilles électrostatiques. Ces lentilles électrostatiques peuvent présenter différentes configurations. On parle en effet de lentilles rondes/monopolaires (un seul potentiel), de lentilles dipolaires (deux potentiels), de lentilles quadrupolaires (quatre potentiels), etc. Notre étude portera plus particulièrement sur les lentilles quadrupolaires.

Le principe de ce projet multidisciplinaire est de créer un code python pour faire la décomposition multipolaire d’un quadrupole électrostatique. Cette étude s’inscrit dans la continuité de deux projets multidisciplinaires précédents. Le premier projet consistait à prendre en main un code déjà existant permettant de modéliser un cas simple de lentilles électrostatiques (les lentilles de Einzel). Le second avait pour but de créer un algorithme permettant d’optimiser ces lentilles. Notre objectif est donc de continuer ce projet en l’adaptant aux lentilles quadrupolaires.

1.4 Livrables

Le premier livrable sera un algorithme de décomposition du potentiel. Le livrable suivant sera un code fonctionnel de modélisation de lentilles quadrupolaires électrostatiques, accompagné d’un manuel d’explication, détaillant le fonctionnement de chaque fonction, des classes et leur utilisation. Dans un dernier temps, nous fournirons une comparaison des résultats de notre code avec ceux du quadrupôle d’Okayama, dont nous connaissons déjà les résultats. De plus, notre code étant open-source, nous fournirons un dépôt GitHub regroupant le code et toute la documentation associée pour qu’il puisse être utilisé.



Chapitre 2

Travail Réalisé

2.1 Prérequis mathématique

2.1.1 Principe de moindre action

Le commencement de ce raisonnement est le principe de moindre action qui s'exprime comme

$$\delta S = \int_{S_0}^{S_1} \vec{p} \frac{\vec{r}}{dS} ds = 0$$

On peut alors montrer que

$$\delta S = \int_{S_0}^{S_1} \mu dz = 0$$

Comme le quadrupole que nous utilisons est composé UNIQUEMENT de lentilles électrostatiques, on peut écrire

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$$

avec μ l'indice optique, ϕ le potentiel électrostatique et x' et y' les dérivées de x et y par rapport à z .

2.1.2 Equations d'Euler-Lagrange

Les équations d'Euler-Lagrange sont des équations équivalentes au principe de moindre action, sous forme différentielle. On peut ainsi noter

$$\frac{\partial \mu}{\partial x} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) = 0$$

$$\frac{\partial \mu}{\partial y} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial y'} \right) = 0$$

2.1.3 Décomposition du potentiel

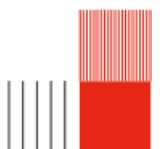
Une fois les équations d'Euler-Lagrange exprimées, il est utile de décomposer le potentiel ϕ afin de séparer les solutions de ces équations en fonction de leur dépendance aux différents ordres de x et y .

On peut ainsi décomposer ce potentiel en série de Fourier et en série de Taylor.

Dans le cadre de notre étude, c'est-à-dire dans le cadre d'un quadrupole, on observe une 2π périodicité du potentiel selon θ . On peut donc décomposer le potentiel comme une série de Fourier. Celle-ci, de part sa périodicité, représente donc les symétries du système. On obtient alors

$$\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \phi_{\nu}(\mathbf{r}, \theta, z)$$

avec



$$\phi_\nu(\mathbf{r}, \theta, z) = a_\nu \cos(\nu\theta) + b_\nu \sin(\nu\theta)$$

Ainsi, le potentiel $\phi(\mathbf{r}, \theta, z)$ est composé d'une somme de fonctions dépendantes de $\mathbf{r}$, θ et z . On remarque alors que les variables $(\mathbf{r}, z)$ et θ sont séparées sous la forme suivante :

$$\phi_\nu(\mathbf{r}, \theta, z) = f(\mathbf{r}, z) \times g(\theta)$$

Il est nécessaire que le potentiel électrostatique satisfasse l'équation de Laplace $\Delta\phi(\vec{r}) = 0$. On peut exprimer cette même équation en coordonnées cylindriques comme

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial \phi}{\partial r} \right) + \frac{1}{r^2} \frac{\partial^2}{\partial \theta^2} + \frac{1}{r^2} \frac{\partial^2}{\partial z^2} = 0$$

En injectant $\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \phi_\nu(\mathbf{r}, \theta, z)$ dans l'équation précédente, on obtient l'équation suivante

$$\sum_{\nu=0}^{\infty} \left(\cos(\nu\theta) \left(\frac{\partial^2 a_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial a_\nu}{\partial r} + \frac{\partial^2 a_\nu}{\partial z^2} - a_\nu \frac{\nu^2}{r^2} \right) + \sin(\nu\theta) \left(\frac{\partial^2 b_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial b_\nu}{\partial r} + \frac{\partial^2 b_\nu}{\partial z^2} - b_\nu \frac{\nu^2}{r^2} \right) \right) = 0$$

On peut alors montrer que cette équation peut être résolue $\forall \mathbf{r}$, si on vérifie

$$\frac{\partial^2 a_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial a_\nu}{\partial r} + \frac{\partial^2 a_\nu}{\partial z^2} - a_\nu \frac{\nu^2}{r^2} = 0$$

et

$$\frac{\partial^2 b_\nu}{\partial r^2} + \frac{1}{r} \frac{\partial b_\nu}{\partial r} + \frac{\partial^2 b_\nu}{\partial z^2} - b_\nu \frac{\nu^2}{r^2} = 0$$

Il est maintenant utile de développer les fonctions $a_\nu(\mathbf{r}, z)$ et $b_\nu(\mathbf{r}, z)$ en série de Taylor, dans le but de séparer la dépendance des variables $\mathbf{r}$ et z . Dans le cadre de notre étude, on considère des conditions paraxiales, c'est-à-dire des conditions où les rayons sont proches de l'axe optique. On effectue donc le développement en série de Taylor pour $\mathbf{r} = 0$.

Rappel : Une série de Taylor $f(x)$ en $x = b$ s'exprime comme $f(x) = \sum_{n=0}^{\infty} a_n (x - b)^n$.

On peut ainsi écrire

$$A_\nu(\mathbf{r}, z) = \sum_{i=0}^{\infty} a_{i\nu}(z) \times (\mathbf{r})^{\nu+i}$$

et

$$B_\nu(\mathbf{r}, z) = \sum_{i=0}^{\infty} b_{i\nu}(z) \times (\mathbf{r})^{\nu+i}$$

On pose alors $A_{\nu 0}(z) = \Phi_{\nu \mathbf{r}}$ et $B_{\nu 0}(z) = \Phi_{\nu i}$, les composantes de Fourier du potentiel sur l'axe. On peut alors insérer les coefficients de la série de Taylor dans les composantes de la série de Fourier. On obtient alors

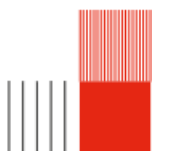
$$A_\nu(\mathbf{r}, z) = \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \Phi_{\nu \mathbf{r}}^{[2\lambda]}(z)$$

et

$$B_\nu(\mathbf{r}, z) = \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \Phi_{\nu i}^{[2\lambda]}(z)$$

On peut alors écrire le potentiel $\phi(\mathbf{r}, \theta, z)$ comme une somme polynomiale avec les dérivées du potentiel pris sur l'axe optique comme coefficients des polynômes. On a alors

$$\phi(\mathbf{r}, \theta, z) = \sum_{\nu=0}^{\infty} \sum_{\lambda=0}^{\infty} \frac{(-1)^\lambda \times (\nu!)}{4^\lambda \times \lambda! \times (\lambda + \nu)!} \times \mathbf{r}^{\nu+2\lambda} \times \left(\Phi_{\nu \mathbf{r}}^{[2\lambda]}(z) \times \cos(\nu\theta) + \Phi_{\nu i}^{[2\lambda]}(z) \times \sin(\nu\theta) \right)$$



2.2 Décomposition des différentes contributions

Une fois le potentiel $\phi(\mathbf{r}, \theta, z)$ décomposé, on peut l'injecter dans l'équation

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)} = \sqrt{\phi} \sqrt{1 + x'^2 + y'^2}$$

afin de déterminer les différentes contributions qui composent l'indice optique.

Cependant, on remarque que l'indice optique ν dépend à l'ordre 2 des dérivées de x et y . Dans la démarche de décomposer les différentes contributions de l'indice optique, il est nécessaire d'effectuer un développement limité de

$$\sqrt{1 + x'^2 + y'^2}$$

On pose $X = x'^2 + y'^2$ afin d'effectuer un changement de variable.

Rappel : Le développement limité d'une racine carrée est $\sqrt{1 + X} = 1 + \frac{x}{2} - \frac{x^2}{8} + \dots$

On obtient ainsi les différentes contributions de l'indice optique $\mu = \mu_0 + \mu_1 + \mu_2 + \mu_3 + \mu_4 + \mu_5 + \dots$

Les composantes μ_0 , μ_1 et μ_2 composent la partie paraxiale de l'équation tandis que les termes d'ordres supérieurs correspondent aux aberrations. μ_0 correspond à l'axe optique du système. μ_2 correspond au paraxial du système optique.

On note que la paraxialité a une dépendance à l'ordre 2 des variables x et y car en optique, le front d'onde est sphérique. Le front d'onde est une surface d'égale phase d'une onde, c'est-à-dire que les points qui composent cette sphère ont mis le même temps de parcours depuis la source. Ainsi, dans des conditions parfaites, où les aberrations de front d'onde n'existent pas, le front d'onde est parfaitement sphérique, ce qui explique la dépendance d'ordre 2 des variables x et y .

Lorsque le front d'onde n'est plus parfaitement sphérique, autrement dit quand le système connaît une perturbation, tous les points du front d'onde ne vont plus focaliser au même endroit. On obtient ainsi des aberrations. Ces dernières correspondent à tous les termes d'ordres supérieurs à 2 de la somme infinie de ν .

2.3 Reconstruction de la décomposition du potentiel

2.3.1 Extraction du potentiel

A l'aide de la bibliothèque Python BEMPP (*BoundaryElementMethodPythonPackage*), on peut extraire le potentiel $\phi(\mathbf{r}, \theta, z)$ en tout point de l'espace. Comme vu dans la partie précédente, il est important de décomposer le potentiel afin de pouvoir séparer la contribution des différents ordres. Une fois le potentiel extrait, il faut donc trouver les différentes composantes de la décomposition du potentiel.

2.3.2 Composantes multipolaires

Equations

On peut retrouver les composantes multipolaires à l'aide de la formule

$$\Phi_\nu = \frac{2 - \delta_{0\nu}}{\nu!} \left(\frac{\partial^\nu \phi}{\partial \bar{w}^\nu} \right) \Big|_{w=0}.$$

où ν est l'ordre du terme multipolaire et $\delta_{0\nu}$ le symbole de Kronecker.

Pour un développement limité à l'ordre N en $\mathbf{r}$, les valeurs admissibles de λ sont déterminées par la condition

$$2\lambda + \nu \leq N$$

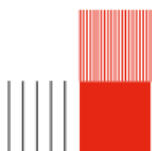
Dans le cadre de ce travail, on se limite à $N = 4$ et $\nu \leq 2$, ce qui conduit aux valeurs suivantes : $\lambda = 0, 1, 2$ pour $\nu = 0$ et $\lambda = 0, 1$ pour $\nu = 1, 2$.

On peut alors calculer les différents ordres des composantes multipolaires :

Cas $\nu = 0$ (monopôle)

On a $\delta_{00} = 1$, d'où

$$\Phi_0(z) = \frac{2 - \delta_{00}}{0!} \frac{\partial^0 \phi}{\partial \bar{w}^0} \Big|_{w=0} = \phi(x, y, z) \Big|_{w=0}.$$



Cas $\nu = 1$ (dipôle)

On a $\delta_{01} = 0$, d'où

$$\Phi_1(z) = \frac{2 - \delta_{01}}{1!} \left(\frac{\partial \phi}{\partial \bar{w}} \right) \Big|_{w=0} = 2 \frac{\partial \phi}{\partial \bar{w}} \Big|_{w=0}.$$

Cas $\nu = 2$ (quadrupôle)

On a $\delta_{02} = 0$, d'où

$$\Phi_2(z) = \frac{2 - \delta_{02}}{2!} \frac{\partial^2 \phi}{\partial \bar{w}^2} \Big|_{w=0} = \frac{1}{2} \frac{\partial^2 \phi}{\partial \bar{w}^2} \Big|_{w=0}.$$

Cas $\nu = 3$ (hexapôle)

On a $\delta_{03} = 0$, d'où

$$\Phi_3(z) = \frac{2 - \delta_{03}}{3!} \frac{\partial^3 \phi}{\partial \bar{w}^3} \Big|_{w=0} = \frac{1}{3} \frac{\partial^3 \phi}{\partial \bar{w}^3} \Big|_{w=0}.$$

Cas $\nu = 4$ (octopôle)

On a $\delta_{04} = 0$, d'où

$$\Phi_4(z) = \frac{2 - \delta_{04}}{4!} \frac{\partial^4 \phi}{\partial \bar{w}^4} \Big|_{w=0} = \frac{1}{12} \frac{\partial^4 \phi}{\partial \bar{w}^4} \Big|_{w=0}.$$

Cas général

On obtient finalement avec $\bar{w} = x - iy$ et $\frac{\partial}{\partial \bar{w}} = \frac{1}{2} \left(\frac{\partial}{\partial x} + i \frac{\partial}{\partial y} \right)$ que

$$\Phi_0(z) = \phi(x, y, z) \Big|_{x=y=0},$$

$$\Phi_1(z) = \left(\frac{\partial \phi}{\partial x} + i \frac{\partial \phi}{\partial y} \right) \Big|_{x=y=0},$$

$$\Phi_2(z) = \frac{1}{8} \left(\frac{\partial^2 \phi}{\partial x^2} - \frac{\partial^2 \phi}{\partial y^2} - 2i \frac{\partial^2 \phi}{\partial x \partial y} \right) \Big|_{x=y=0},$$

$$\Phi_3(z) = \frac{1}{24} \left(\frac{\partial^3 \phi}{\partial x^3} - 3 \frac{\partial^3 \phi}{\partial x \partial y^2} - 3i \frac{\partial^3 \phi}{\partial x^2 \partial y} + i \frac{\partial^3 \phi}{\partial y^3} \right) \Big|_{x=y=0},$$

$$\Phi_4(z) = \frac{1}{192} \left(\frac{\partial^4 \phi}{\partial x^4} - 6 \frac{\partial^4 \phi}{\partial x^2 \partial y^2} + \frac{\partial^4 \phi}{\partial y^4} - 4i \frac{\partial^4 \phi}{\partial x^3 \partial y} + 4i \frac{\partial^4 \phi}{\partial x \partial y^3} \right) \Big|_{x=y=0}.$$

Ici, $\frac{\partial^\nu \phi}{\partial \bar{w}^\nu}$ représente la dérivée d'ordre ν du potentiel par rapport à la coordonnée transverse w évaluée sur l'axe central $w = 0$.

Comme dans le cas de notre code, on choisit le référentiel (x,y) tel qu'un cosinus décrit le quadrupôle.

On peut donc écrire :

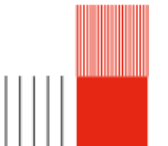
$$\Phi_0(z) = \phi(x, y, z) \Big|_{x=y=0}$$

$$\Phi_1(z) = \left(\frac{\partial \phi}{\partial x} \right) \Big|_{x=y=0}$$

$$\Phi_2(z) = \frac{1}{8} \left(\frac{\partial^2 \phi}{\partial x^2} - \frac{\partial^2 \phi}{\partial y^2} \right) \Big|_{x=y=0}$$

$$\Phi_3(z) = \frac{1}{24} \left(\frac{\partial^3 \phi}{\partial x^3} - 3 \frac{\partial^3 \phi}{\partial x \partial y^2} \right) \Big|_{x=y=0}$$

$$\Phi_4(z) = \frac{1}{192} \left(\frac{\partial^4 \phi}{\partial x^4} - 6 \frac{\partial^4 \phi}{\partial x^2 \partial y^2} + \frac{\partial^4 \phi}{\partial y^4} \right) \Big|_{x=y=0}$$



Applications dans BEMPP

Les dérivées d'ordre supérieur du potentiel par rapport à la coordonnée transverse w , à savoir

$$\frac{\partial \phi}{\partial \bar{w}} \quad \text{et} \quad \frac{\partial^2 \phi}{\partial \bar{w}^2},$$

sont obtenues directement à l'aide des dérivées de BEMPP.

- le gradient du potentiel via `single_layer_gradient`, correspondant à $\nabla \phi$
- la dérivée seconde via `single_layer_2nd_deriv`.

On a pour le terme dipolaire

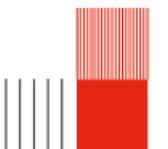
$$\begin{aligned} E_eval[0, :] &= -\frac{\partial \phi}{\partial x} \\ E_eval[1, :] &= -\frac{\partial \phi}{\partial y} \\ E_eval[2, :] &= -\frac{\partial \phi}{\partial x} \end{aligned}$$

On a pour le terme quadripolaire :

$$\begin{aligned} D2_eval[0, :] &= \frac{\partial^2 \phi}{\partial x^2} \\ D2_eval[3, :] &= \frac{\partial^2 \phi}{\partial y^2} \\ D2_eval[1, :] &= \frac{\partial^2 \phi}{\partial x \partial y} \\ D2_eval[5, :] &= \frac{\partial^2 \phi}{\partial z^2} \end{aligned}$$

On a pour le terme hexapolaire :

$$\begin{aligned} D3_eval[0, :] &= \frac{\partial^3 \phi}{\partial x^3} \\ D3_eval[4, :] &= \frac{\partial^3 \phi}{\partial x \partial y^2} \\ D3_eval[1, :] &= \frac{\partial^3 \phi}{\partial x^2 \partial y} \\ D3_eval[6, :] &= \frac{\partial^3 \phi}{\partial y^3} \end{aligned}$$



On a pour le terme octopolaire

$$\begin{aligned}
D4_eval[0, :] &= \frac{\partial^4 \phi}{\partial x^4} \\
D4_eval[4, :] &= \frac{\partial^4 \phi}{\partial x^2 \partial y^2} \\
D4_eval[10, :] &= \frac{\partial^4 \phi}{\partial y^4} \\
D4_eval[1, :] &= \frac{\partial^4 \phi}{\partial x^3 \partial y} \\
D4_eval[7, :] &= \frac{\partial^4 \phi}{\partial x \partial y^3}
\end{aligned}$$

Résultats attendus

Fonctions de fit

Afin de vérifier résultats que nous obtenons, il est utile de comparer ces derniers aux résultats publiés par S.Okayama dans son papier ; [S. Okayama et H. Kawakatsu, «*Potential distribution and focal properties of electrostatic quadrupole lenses* », *Journal of Physics E : Scientific Instruments*, vol. 11, no 3, p. 211, 1978]. Dans cet article, des fonctions de fit sont utilisées. Il est donc intéressant d'exprimer ces mêmes fonctions dans le cadre de notre étude afin de pouvoir s'assurer de la fiabilité des résultats. Dans le papier d'Okayama, les fonctions de fit sont exprimées comme :

$$\Phi_0(Z) = V_A k_0(Z)$$

$$\Phi_2(Z) = \frac{V_Q}{a^2} k_2(Z)$$

$$\Phi_4(Z) = \frac{V_A}{a^4} k_4(Z)$$

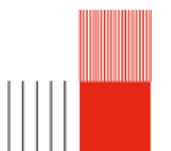
avec $\Phi_0(Z)$, $\Phi_2(Z)$ et $\Phi_4(Z)$ les composantes du potentiel sur l'axe, $k_0(z)$, $k_2(z)$ et $k_4(z)$ les fonctions de fit, V_A le potentiel des apertures et V_Q la valeur absolue du potentiel des quadrupoles.

De plus, les fonctions de fit sont définies comme

$$k_0(Z) = a_0 \exp\left(-\frac{Z^2}{b^2}\right)$$

$$\begin{cases} k_2(Z) = 1 & \text{pour } (-Z_0 \leq Z \leq Z_0) \\ k_2(Z) = \exp\left(-\frac{(Z-Z_0)^2}{b^2}\right) & \text{pour } (Z < -Z_0, Z > Z_0) \end{cases}$$

$$\begin{cases} k_4(Z) = \frac{a_4}{\left(1 + \frac{Z^2}{b_1^2}\right)^2} & \text{pour } (Z \leq 0) \\ k_4(Z) = a_4 \exp\left(-\frac{Z^2}{b_2^2}\right) & \text{pour } (Z > 0) \end{cases}$$



2.4 Trajectoire

2.4.1 Principe de fermat

Pour obtenir l'équation de la trajectoire on peut partir du principe de fermat

$$\delta \int \mu ds = 0$$

Dans le cas de l'optique des particules chargés l'indice optique s'écrit comme

$$\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$$

On a donc notre Lagrangien de notre systeme

$$L(x, y, x', y', z) = \sqrt{\phi(1 + x'^2 + y'^2)}$$

2.4.2 Cas $\nu = 2$, quadrupole

Si on prend le cas $\nu = 2$, notre potentiel devient :

$$\phi_2(r, \theta, z) = r^2 \times (\phi_2(z) \times \cos(2\theta)) + \frac{r^4}{12} \times (\phi_2''(z) \times \cos(4\theta))$$

avec $r^2 \cos(2\theta) = x^2 - y^2$

On a donc dans l'approximation paraxiale $\phi_2(r, \theta, z) = \phi_2(z)(x^2 - y^2)$ et donc dans la cas paraxiale :

$$\phi(x, y, z) = \phi_0(z) - \frac{1}{4}\phi_0''(z)(x^2 + y^2) + \phi_2(z)(x^2 - y^2)$$

On peut ensuite injecter ce potentiel dans notre indice optique $\mu = \sqrt{\phi(1 + x'^2 + y'^2)}$, après développement on retrouve les equations d'euler lagrange dans le cas d'un quadrupole)

On peut écrire notre indice optique comme $\mu = \sqrt{\phi(1 + \epsilon)}$ avec $\epsilon : x'^2 + y'^2$. d'après Taylor on a $\sqrt{1 + \epsilon} = 1 + \frac{\epsilon}{2}$

En remplaçant ϵ on peut donc développer l'indice optique comme

$$\mu = \sqrt{\phi} \times \sqrt{1 + x'^2 + y'^2} = \sqrt{\phi} \times (1 + \frac{1}{2}(x'^2 + y'^2)) = \sqrt{\phi} + \frac{\sqrt{\phi}}{2}(x'^2 + y'^2)$$

En remplaçant ϕ par son approximation paraxiale on a :

$$\mu \approx \sqrt{\phi_0 - \frac{1}{4}\phi_0''(x^2 + y^2) + \phi_2(x^2 - y^2)} + \frac{\sqrt{\phi_0 + \dots}}{2}(x'^2 + y'^2)$$

avec

$$\sqrt{\phi_0 - \frac{1}{4}\phi_0''(x^2 + y^2) + \phi_2(x^2 - y^2)} = \sqrt{\phi_0} \sqrt{1 - \frac{\phi_0''}{4\phi_0}(x^2 + y^2) + \frac{\phi_2}{\phi_0}(x^2 - y^2)}$$

En utilisant $\sqrt{1 + \epsilon} \approx 1 + \frac{\epsilon}{2}$, on obtient :

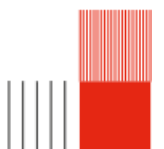
$$\sqrt{\phi_0} \left(1 - \frac{\phi_0''}{8\phi_0}(x^2 + y^2) + \frac{\phi_2}{2\phi_0}(x^2 - y^2) \right) = \sqrt{\phi_0} - \frac{\phi_0''}{8\sqrt{\phi_0}}(x^2 + y^2) + \frac{\phi_2}{2\sqrt{\phi_0}}(x^2 - y^2)$$

$$\mu = \sqrt{\phi_0} - \frac{\phi_0''}{8\sqrt{\phi_0}}(x^2 + y^2) + \frac{\phi_2}{2\sqrt{\phi_0}}(x^2 - y^2) + \frac{\sqrt{\phi_0}}{2}(x'^2 + y'^2)$$

On peut injecter μ dans les equations d'euler lagrange pour retrouver l

$$\frac{\partial \mu}{\partial x} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) = 0$$

$$\frac{\partial \mu}{\partial y} - \frac{d}{dz} \left(\frac{\partial \mu}{\partial y'} \right) = 0$$



Avec :

$$\begin{aligned}\frac{\partial \mu}{\partial x} &= -\frac{\phi_0''}{4\sqrt{\phi_0}}x + \frac{\phi_2}{\sqrt{\phi_0}}x = \left(-\frac{\phi_0''}{4\sqrt{\phi_0}} + \frac{\phi_2}{\sqrt{\phi_0}}\right)x \\ \frac{\partial \mu}{\partial x'} &= \sqrt{\phi_0}x' \\ \frac{d}{dz} \left(\frac{\partial \mu}{\partial x'} \right) &= \sqrt{\phi_0}x'' + \frac{\phi_0'}{2\sqrt{\phi_0}}x'\end{aligned}$$

On obtient finalement l'équation :

$$\left(-\frac{\phi_0''}{4\sqrt{\phi_0}} + \frac{\phi_2}{\sqrt{\phi_0}}\right)x - \left(\sqrt{\phi_0}x'' + \frac{\phi_0'}{2\sqrt{\phi_0}}x'\right) = 0$$

En multipliant par $-\frac{1}{\sqrt{\phi_0}}$, on retrouve la forme :

$$x'' + \frac{\phi_0'}{2\phi_0}x' + \left(\frac{\phi_0''}{4\phi_0} - \frac{\phi_2}{\phi_0}\right)x = 0$$

2.4.3 Equation d'euler lagrange

Il nous reste donc à résoudre les équations différentielles suivantes à l'aide de la méthode Runge Kutta 4 afin d'obtenir les trajectoires

$$\begin{aligned}x''(z) + \frac{\phi_0'(z)}{2\phi_0}x'(z) + \left(\frac{\phi_0''(z)}{4\phi_0(z)} - \frac{\phi_2(z)}{\phi_0(z)}\right)x(z) &= 0 \\ y''(z) + \frac{\phi_0'(z)}{2\phi_0}y'(z) + \left(\frac{\phi_0''(z)}{4\phi_0(z)} + \frac{\phi_2(z)}{\phi_0(z)}\right)y(z) &= 0\end{aligned}$$

On peut trouver les solutions tq :

rayon principal : $x(z_0) = 1, x'(z_0) = 0$

rayon marginal : $x(z_0) = 0, x'(z_0) = 1$

2.4.4 Résolution équation diff(cas sans polariser les apertures)

Dans les deux cas à résoudre une équation différentielle de type

$$x''(z) + \omega_0^2 x = 0$$

On résout cette équation avec la méthode Runge Kutta 4 avec les conditions initiales :

Pour le rayon principal :

$$x(z_0) = 1 \text{ et } x'(z_0) = 0$$

Pour le rayon marginal :

$$x(z_0) = 0 \text{ et } x'(z_0) = 1$$

On peut vérifier le résultat analytiquement

2.4.5 Vérification des résultats

Cas $\omega_0 > 0$

On vérifie les résultats analytiquement

Dans le cas où $\omega_0^2 > 0$, on a (focalisation direction x) :

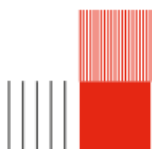
$$x''(z) + \omega_0^2 x(z) = 0.$$

La solution générale est alors :

$$x_m(z) = A \cos(\omega_0(z - z_0)) + B \sin(\omega_0(z - z_0)).$$

On considère le cas du rayon marginal :

$$x(z_0) = 0, \quad x'(z_0) = 1.$$



$$x(z_0) = A = 0.$$

$$x'(z) = -\omega_0 A \sin(\omega_0(z - z_0)) + \omega_0 B \cos(\omega_0(z - z_0)).$$

Ainsi :

$$x'(z_0) = \omega_0 B = 1, \quad \Rightarrow \quad B = \frac{1}{\omega_0}.$$

On obtient finalement la solution analytique du rayon marginal :

$$x_m(z) = \frac{1}{\omega_0} \sin(\omega_0(z - z_0))$$

On considère le cas du rayon principal :

$$x(z_0) = 1, \quad x'(z_0) = 0.$$

$$x(z_0) = A = 1.$$

$$x'(z) = -\omega_0 A \sin(\omega_0(z - z_0)) + \omega_0 B \cos(\omega_0(z - z_0)).$$

$$x'(z_0) = \omega_0 B = 0, \quad \Rightarrow \quad B = 0.$$

On obtient finalement la solution du rayon principal :

$$x_p(z) = \cos(\omega_0(z - z_0))$$

Cas $\omega_0 < 0$

Dans le cas où ω_0 est inférieur à 0 (dans la direction y) on obtient la trajectoire de défocalisation on a

$$y''(z) + \omega_0^2 y(z) = 0.$$

La solution générale est alors :

$$y_m(z) = A \cosh(\omega_0(z - z_0)) + B \sinh(\omega_0(z - z_0)).$$

On considère le cas du rayon marginal :

$$y(z_0) = 0, \quad y'(z_0) = 1.$$

$$y(z_0) = A = 0.$$

$$y'(z) = -\omega_0 A \sinh(\omega_0(z - z_0)) + \omega_0 B \cosh(\omega_0(z - z_0)).$$

Ainsi :

$$y'(z_0) = \omega_0 B = 1, \quad \Rightarrow \quad B = \frac{1}{\omega_0}.$$

On obtient finalement la solution analytique du rayon marginal :

$$y_m(z) = \frac{1}{\omega_0} \sinh(\omega_0(z - z_0))$$

On considère le cas du rayon principal :

$$y(z_0) = 1, \quad y'(z_0) = 0.$$

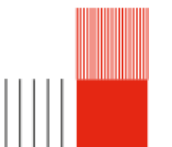
$$y(z_0) = A = 1.$$

$$y'(z) = -\omega_0 A \sinh(\omega_0(z - z_0)) + \omega_0 B \cosh(\omega_0(z - z_0)).$$

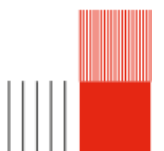
$$y'(z_0) = \omega_0 B = 0, \quad \Rightarrow \quad B = 0.$$

On obtient finalement la solution du rayon principal :

$$y_p(z) = \cosh(\omega_0(z - z_0))$$



2.5 Document Okayama



Chapitre 3

Notre production

3.1 Structure globale du code

Comme évoqué dans l'introduction, le rendu final de ce projet est un code Python permettant la modélisation du quadrupole d'Okayama et du comportement des rayons d'électrons à l'intérieur de celui-ci. Ce code étant assez long, nous avons décidé de le fragmenter en plusieurs codes en utilisant la programmation orientée objet. La programmation orientée objet tend à représenter un système en utilisant différents objets qui représentent chaque aspect du système. Le code est structuré en trois étage. Le premier étage est composé des objets qui vont définir toutes les variables et les fonctions que nous utilisons dans le code. Il regroupe les codes suivants : *Data.py*, *Geometry.py*, *Field_calculation.py*, *Extraction_data.py*, *Multipolar_decomposition.py*, *Fit_functions.py*, *Graphs.py* et *paraxial.py*. Le deuxième étage est composé d'un unique programme intitulé *Main_functions.py* qui crée des objets qui regroupent toutes les objets de l'étage précédent et permet de les appeler dans l'ordre. Le troisième étage, quant à lui, est l'étage utilisateur. Il appelle les objets créés dans l'étage précédent et permet de les exécuter.

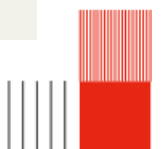
3.2 Codes utilisateur

Cette partie correspond aux deux codes composant le troisième étage. C'est l'interface utilisateur du programme. Nous avons décidé de séparer ce code en deux codes. Le premier code permet d'extraire le potentiel et de l'exporter dans un fichier, étape assez chronophage. Le deuxième code permet d'importer les données extraites par le premier code et de les traiter.

3.2.1 Extraction du potentiel

Ce code permet de créer la géométrie du quadrupole que l'on cherche à modéliser, le mailler et exporter le maillage. Ensuite, le code permet d'extraire le potentiel en utilisant le maillage et la méthode des éléments de frontière (BEM) puis exporte ces données dans un fichier.

```
1 from Main_functions import Potential_extraction
2 from Data import Data
3 import os
4 from bempt.cl.core import opencl_kernels
5 from Field_calculation import Calculation_field
6 import gc
7
8 opencl_kernels.show_available_platforms_and_devices()
9 opencl_kernels.set_default_cpu_device(0,1)
10
11 os.environ['PYOPENCL_COMPILER_OUTPUT']='1'
12 os.environ['PYOPENCL_NO_CACHE'] = '1'
13
14 #OUTPUT_DIR = r"C:\Users\zoeno\OneDrive - INSA Toulouse\Documents\INSA\4GP\Projet
15   multi\projet-multi-code\Quadrupole_model\Files"
16 OUTPUT_DIR =
17   r"C:\Users\llamm\OneDrive\Documents\Projet\BEMPP\okayama\projet_multi\projet-multi-code\Quadrupole_model\Files"
18   "-0.0299087*Va, -0.18808*Va, +0.18808*Va" #tension qui fit
19
20 # Dico pour faire étape par étape
```



```

19 quad = {
20     "aperture1": [0, 0, 1, 0],
21     "aperture2": [0, 0, 0, 1],
22     "quad13":    [1, 0, 0, 0],
23     "quad24":    [0, 1, 0, 0]
24 }
25
26
27 # Générer le maillage une fois
28 data_initial = Data(4, 5, 21, 19, 2, 15, 2, 13, 3.4934, 0, 0, 0, 0, 0, 1.5, 0.9, 2, 30, OUTPUT_DIR)
29 fun_initial = Potential_extraction(data_initial, True, "temp.npz")
30 fun_initial.mesh()
31 saved_group_ids = data_initial.group_id
32
33 # On fait élément par élément
34 for element, tensions in quad.items():
35     vq13, vq24, va1, va2 = tensions
36
37     file_name = f"{element}.npz"
38
39     data = Data(4, 5, 21, 19, 2, 15, 2, 13, 3.4934, vq13, vq24, va1, va2, 0, 1.5, 0.9, 2, 30, OUTPUT_DIR)
40     data.group_id = saved_group_ids
41
42     fun = Potential_extraction(data, False, file_name)
43
44     fun.potential_extraction() # Résolution BEM
45     fun.potential_visualisation()
46
47     fun.graph_potential_axis()
48     del fun
49     gc.collect()
50
51 #data = Data(4, 5, 21, 19, 2, 15, 2, 13, 3.4934, 1, -0.18808*Va, +0.18808*Va, 0, Va, 0.9, 3, 20, OUTPUT_DIR)

```

3.2.2 Exploitation des données

Ce code permet d'importer les données extraites du codes précédents. Il permet ensuite de les traiter notamment en effectuant la décomposition du potentiel, en créant des fonctions de fit et en retrouvant les équations paraxiales.

3.3 Programme principal

Ce programme comporte les codes du premier étage, soit les codes permettant la définition de toutes les variables et l'appel de toutes les fonctions nécessaires ; ce sont les fondements de notre code.

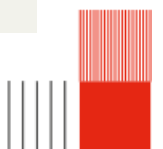
3.3.1 Data

Ce code permet de définir toutes les variables que l'on utilise à travers ce programme, on peut notamment citer les dimensions du quadrupole, les tensions appliquées aux différents éléments du quadrupole et la taille du maillage.

```

1 class Data:
2     """
3     #1 - Distance between the shield ant the aperture
4     #2 - Distance between the aperture and the cylinders (d in Okayama's paper)
5     #3 - Exterior radius of the shield
6     #4 - Inside radius of the shield
7     #5 - Thickness of the shield
8     #6 - Radius of the aperture
9     #7 - Thickness of the aperture
10    #8 - Length of the cylinder (l in Okayama's paper)
11    #9 - Radius of the elements around the axis (a in Okayama's paper)
12    """
13    def __init__(self,
14        dist_shield_apert: int, dist_apert_quad: int,
15        radius_ext_shield: int, radius_in_shield: int, thickness_shield: int,
16        radius_apert: int, thickness_apert: int,
17        length_cylinder: int,

```



```

18         radius_axis: int,
19         pot_electrode13: int, pot_electrode24 :int, pot_apert1: int, pot_apert2: int, pot_shield:
20         int, pot_acceleration: int,
21         MeshSizeMin: int, MeshSizeMax: int, MeshSizeFromCurvature: int,
22         output_dir: str) -> None:
23     self.dist_shield_apert = dist_shield_apert
24     self.dist_apert_quad = dist_apert_quad
25
26     self.radius_ext_shield = radius_ext_shield
27     self.radius_in_shield = radius_in_shield
28     self.thickness_shield = thickness_shield
29
30     self.radius_apert = radius_apert
31     self.thickness_apert = thickness_apert
32
33     self.length_cylinder = length_cylinder
34
35     self.radius_axis = radius_axis
36
37     self.pot_electrode13 = pot_electrode13 #quadrupole barreaux 1 et 3
38     self.pot_electrode24 = pot_electrode24 #quadrupole barreaux 2 et 4
39     self.pot_apert1 = pot_apert1 #first aperture (pour le champs rond)
40     self.pot_apert2 = pot_apert2 #second aperture
41     self.pot_shield = pot_shield #shield (0)
42     self.pot_acceleration = pot_acceleration #ion acceleration potential
43
44     self.MeshSizeMin = MeshSizeMin
45     self.MeshSizeMax = MeshSizeMax
46     self.MeshSizeFromCurvature = MeshSizeFromCurvature
47
48     self.output_dir = output_dir
49
50     self.group_id = None
51
52     self.total_length = 2*self.dist_shield_apert + 2*self.thickness_shield + 2*self.dist_apert_quad +
53     2*self.thickness_apert + self.length_cylinder
54
55     self.coord_cylinder_x_or_y = 2*self.radius_axis
56     self.coord_cylinder_z = self.thickness_shield + self.dist_shield_apert + self.thickness_apert +
57     self.dist_apert_quad
58
59     self.coord_apert_z1 = self.thickness_shield + self.dist_shield_apert
60     self.coord_apert_z2 = self.thickness_shield + self.dist_shield_apert + self.thickness_apert +
61     2*self.dist_apert_quad + self.length_cylinder
62
63     self.start_shield1 = 0
64     self.end_shield1 = self.start_shield1 + self.thickness_shield
65     self.start_apert1 = self.end_shield1 + self.dist_shield_apert
66     self.end_apert1 = self.start_apert1 + self.thickness_apert
67     self.start_cyl = self.end_apert1 + self.dist_apert_quad
68     self.end_cyl = self.start_cyl + self.length_cylinder
69     self.start_apert2 = self.end_cyl + self.dist_apert_quad
70     self.end_apert2 = self.start_apert2 + self.thickness_apert
71     self.start_shield2 = self.end_apert2 + self.dist_shield_apert
72     self.end_shield2 = self.start_shield2 + self.thickness_shield

```

3.3.2 Geometry

Ce code définit plusieurs classes qui permettent de modéliser les différents éléments qui composent un quadrupole (aperture, électrodes, shield). Une autre classe permet de générer un maillage sur la surface du quadrupole. Ce maillage est ensuite exporté et utilisé afin d'appliquer la méthode des éléments de frontière (BEM). La génération du maillage se fait à l'aide de la bibliothèque Python intitulée gmsh.

```

1 import gmsh
2 import os
3 from Data import Data
4
5 #Definition of classes to describe the geometry
6 class Cylinder:
7     """
8     Represents a cylindrical electrode in the GMSH geometry.
9     """

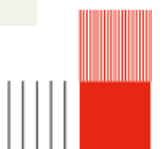
```



```

10 def __init__(self, length: int, radius: int, coord_x: int, coord_y: int, coord_z: int) -> None:
11     """
12     Initialize cylinder parameters.
13
14     Args:
15         length: Length of the cylinder along the Z-axis.
16         radius: Radius of the cylinder.
17         coord_x, coord_y, coord_z: Starting position of the cylinder.
18     """
19     self.length = length
20     self.radius = radius
21     self.coord_x = coord_x
22     self.coord_y = coord_y
23     self.coord_z = coord_z
24
25     self.cyl_tag = None
26     self.cyl_surf = None
27
28
29 def add(self) -> None:
30     """Add the cylinder to the OpenCASCADE model and extract its surface loop."""
31     self.cyl_tag = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0,
32     self.length, self.radius)
33     self.cyl_surf = gmsh.model.occ.get_surface_loops(self.cyl_tag)[1][0]
34
35 class Aperture:
36     """
37     Represents an aperture in the GMSH geometry.
38     """
39     def __init__(self, radius_ext: int, radius_in: int, thickness: int, coord_x: int, coord_y: int,
40     coord_z: int) -> None:
41         """
42         Initialize aperture parameters.
43
44         Args:
45             radius_ext: Outer radius of the aperture disc.
46             radius_in: Inner radius (hole) of the aperture.
47             thickness: Thickness of the disc.
48         """
49         self.radius_ext = radius_ext
50         self.radius_in = radius_in
51         self.thickness = thickness
52         self.coord_x = coord_x
53         self.coord_y = coord_y
54         self.coord_z = coord_z
55
56         self.apert_tag = None
57         self.apert_surf = None
58
59     def add(self) -> None:
60         """Create the aperture """
61         aperture_out = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0,
62         self.thickness, self.radius_ext)
63         aperture_in = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0,
64         self.thickness, self.radius_in)
65         apert_vol , _ = gmsh.model.occ.cut([(3,aperture_out)],[(3,aperture_in)])
66         self.apert_tag=apert_vol[0][1]
67         self.apert_surf = gmsh.model.occ.get_surface_loops(self.apert_tag)[1][0]
68
69 class Shield:
70     """
71     Represents the shield to fixe the potential ot 0
72     """
73     def __init__(self, length: int, radius_ext: int, radius_in: int, radius_hole: int, thickness: int,
74     coord_x: int, coord_y: int, coord_z: int) -> None:
75         self.length = length
76         self.radius_ext = radius_ext
77         self.radius_in = radius_in
78         self.radius_hole = radius_hole
79         self.thickness = thickness
80         self.coord_x = coord_x

```



```

78     self.coord_y = coord_y
79     self.coord_z = coord_z
80
81     self.shield_tag = None
82     self.shield_surf = None
83
84
85     def add(self) -> None:
86         shield_in = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z + self.thickness,
87         0, 0, self.length - 2*self.thickness, self.radius_in)
88         shield_out = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0,
89         self.length, self.radius_ext)
90         shield , _ = gmsh.model.occ.cut([(3,shield_out)],[(3,shield_in)])
91         shield_hole1 = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.length -
92         self.thickness, 0, 0, self.thickness, self.radius_hole)
93         shield_hole2 = gmsh.model.occ.addCylinder(self.coord_x, self.coord_y, self.coord_z, 0, 0,
94         self.thickness, self.radius_hole)
95         shield_vol_1, _ = gmsh.model.occ.cut([(3,shield[0][1])],[(3,shield_hole1)])
96         shield_vol_2, _ = gmsh.model.occ.cut([(3,shield_vol_1[0][1])],[(3,shield_hole2)])
97         self.shield_tag=shield_vol_2[0][1]
98         self.shield_surf = gmsh.model.occ.get_surface_loops(self.shield_tag)[1][0]
99
100
101 class Mesh_Generation:
102     """
103     Manages the full GMSH workflow: initialization, geometry construction,
104     physical grouping, and mesh generation.
105     """
106     def __init__(self, data: Data, visual: bool) -> None:
107         """
108         data (Data): Object containing all geometric dimensions and mesh settings.
109         visual (bool): If True, launches the GMSH GUI after generation.
110         """
111         self.data = data
112         self.visual = visual
113
114         self.objects = None
115
116     def initialisation(self) -> None:
117         """Initialize GMSH and create a new model."""
118
119         gmsh.initialize()
120         gmsh.clear()
121         gmsh.model.add("Quadrupole lens")
122
123     def geometry(self) -> None:
124         """Instantiate and add all geometric components (electrodes, apertures, shield)."""
125
126         cylinder1 = Cylinder(self.data.length_cylinder, self.data.radius_axis,
127         self.data.coord_cylinder_x_or_y, 0, self.data.coord_cylinder_z)
128         Cylinder.add(cylinder1)
129
130         cylinder2 = Cylinder(self.data.length_cylinder, self.data.radius_axis,
131         -self.data.coord_cylinder_x_or_y, 0, self.data.coord_cylinder_z)
132         Cylinder.add(cylinder2)
133
134         cylinder3 = Cylinder(self.data.length_cylinder, self.data.radius_axis, 0,
135         self.data.coord_cylinder_x_or_y, self.data.coord_cylinder_z)
136         Cylinder.add(cylinder3)
137
138         cylinder4 = Cylinder(self.data.length_cylinder, self.data.radius_axis, 0,
139         -self.data.coord_cylinder_x_or_y, self.data.coord_cylinder_z)
140         Cylinder.add(cylinder4)
141
142         aperture1 = Aperture(self.data.radius_apert, self.data.radius_axis, self.data.thickness_apert, 0,
143         0, self.data.coord_apert_z1)
144         Aperture.add(aperture1)
145
146         aperture2 = Aperture(self.data.radius_apert, self.data.radius_axis, self.data.thickness_apert, 0,
147         0, self.data.coord_apert_z2)
148         Aperture.add(aperture2)

```

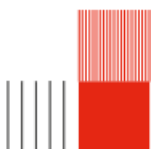


```

141
142
143     shield = Shield(self.data.total_length, self.data.radius_ext_shield, self.data.radius_in_shield,
144 self.data.radius_axis, self.data.thickness_shield, 0, 0, 0)
145     Shield.add(shield)
146
147     self.objects = aperture1, aperture2, cylinder1, cylinder2, cylinder3, cylinder4, shield
148
149 def creation_mesh(self) -> None:
150     """Synchronize the OpenCASCADE internal CAD representation with the GMSH model."""
151     gmsh.model.occ.synchronize()
152
153 def surfaces(self) -> None:
154     """
155     Define physical groups for the surfaces.
156     """
157     aperture1, aperture2, cylinder1, cylinder2, cylinder3, cylinder4, shield = self.objects
158
159     #Outwards orientation of the surfaces' normals
160     gmsh.model.mesh.setOutwardOrientation(aperture1.apert_tag)
161     gmsh.model.mesh.setOutwardOrientation(aperture2.apert_tag)
162     gmsh.model.mesh.setOutwardOrientation(cylinder1.cyl_tag)
163     gmsh.model.mesh.setOutwardOrientation(cylinder2.cyl_tag)
164     gmsh.model.mesh.setOutwardOrientation(cylinder3.cyl_tag)
165     gmsh.model.mesh.setOutwardOrientation(cylinder4.cyl_tag)
166     gmsh.model.mesh.setOutwardOrientation(shield.shield_tag)
167
168     #Adds surfaces on top of the volumes created
169     group_id_apert1 = gmsh.model.addPhysicalGroup(2, aperture1.apert_surf)
170     group_id_apert2 = gmsh.model.addPhysicalGroup(2, aperture2.apert_surf)
171     group_id_cyl1 = gmsh.model.addPhysicalGroup(2, cylinder1.cyl_surf)
172     group_id_cyl2 = gmsh.model.addPhysicalGroup(2, cylinder2.cyl_surf)
173     group_id_cyl3 = gmsh.model.addPhysicalGroup(2, cylinder3.cyl_surf)
174     group_id_cyl4 = gmsh.model.addPhysicalGroup(2, cylinder4.cyl_surf)
175     group_id_shield = gmsh.model.addPhysicalGroup(2, shield.shield_surf)
176
177     self.data.group_id = group_id_apert1, group_id_apert2, group_id_cyl1, group_id_cyl2,
178     group_id_cyl3, group_id_cyl4, group_id_shield
179
180 def mesh(self) -> None:
181     """Configure mesh size and generate the surface mesh."""
182     #Size of the mesh
183     #ATTENTION, MeshSizeMin and MeshSizeMax need to be equal
184     gmsh.option.set_number('Mesh.MeshSizeMin', self.data.MeshSizeMin)
185     gmsh.option.set_number('Mesh.MeshSizeMax', self.data.MeshSizeMax)
186     gmsh.option.set_number('Mesh.MeshSizeFromCurvature', self.data.MeshSizeFromCurvature)
187
188     gmsh.model.mesh.generate(2)
189
190 def finalize(self) -> None:
191     #Creates a file .msh
192     mesh_path = os.path.join(self.data.output_dir, "mesh_quadrapole.msh")
193
194     gmsh.write(mesh_path)
195     print("Mesh saved to:", mesh_path)
196
197     if self.visual == True:
198         #Opens a terminal to see the geometry
199         gmsh.fltk.run()
200
201     gmsh.finalize()

```

Voici une représentation du maillage crée par le code précédent :



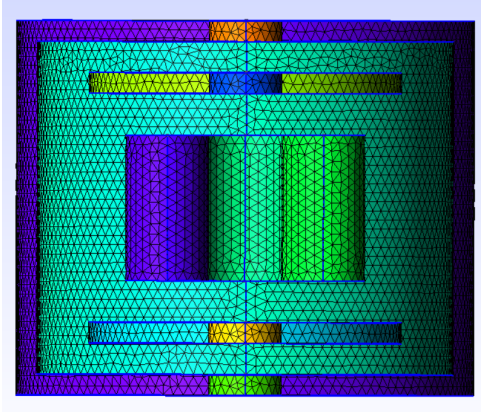


FIGURE 3.1 – Vue en coupe du maillage du quadrupole

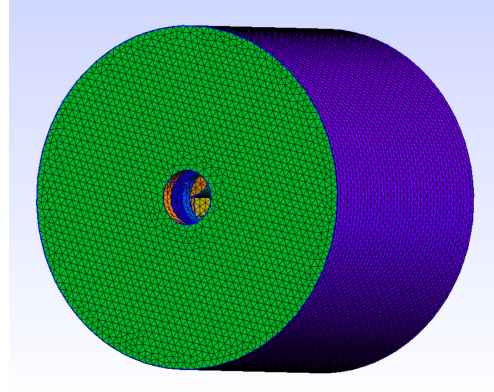


FIGURE 3.2 – Vue du maillage du quadrupole

3.3.3 Field calculation

Ce code permet d'importer le maillage et d'extraire le potentiel électrostatique en tout point du volume en utilisant la méthode des éléments de frontière (BEM). L'application de cette méthode se fait à l'aide de la bibliothèque Python nommée BEMPP (Boundary Element Method Python Package).

```

1 import os
2 import bempp_cl.api as bempp
3 import numpy as np
4 from matplotlib import pylab as plt
5 from Data import Data
6
7 class Calculation_field:
8     """
9     Handles the electrostatic field calculation using the Boundary Element Method (BEM).
10    It manages mesh importation, potential settings, matrix inversion, and
11    derivative calculations for a quadrupole geometry.
12    """
13    def __init__(self, data: Data , file_name : str) -> None:
14
15        """
16        Initialize the calculation field with geometry data and BEM spaces.
17
18        Args:
19            data (Data): Object containing physical dimensions, potentials, and mesh paths.
20        """
21        self.data = data
22        self.file_name = file_name
23
24        #Importation of the mesh
25        self.mesh_path = os.path.join(self.data.output_dir, "mesh_quadrupole.msh")
26        self.grid = bempp.import_grid(self.mesh_path)
27
28        # Functions spaces
29        self.dp0_space = bempp.function_space(self.grid, "DP", 0)
30        self.p1_space = bempp.function_space(self.grid, "P", 1)
31
32        # --- operators ---
33        self.identity = bempp.operators.boundary.sparse.identity(self.p1_space, self.p1_space,
34        self.dp0_space)
35
36        self.dlp = bempp.operators.boundary.laplace.double_layer(self.p1_space, self.p1_space,
37        self.dp0_space)
38
39        self.slp = bempp.operators.boundary.laplace.single_layer(self.dp0_space, self.p1_space,
40        self.dp0_space)
41
42        #Initial our variable to store the results
43
44        self.dirichlet_fun = None
45        self.neumann_fun = None
46        self.u_evaluated = None

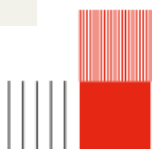
```



```

44     self.points = None
45
46     #Potenital derivative
47
48     self.E_eval = None
49     self.D2_eval = None
50     self.D3_eval = None
51     self.D4_eval = None
52
53
54     def mesh_Importation(self) -> None:
55         plt.clf()
56
57
58     def potentials_settings(self) -> None:
59         """
60         Map physical potentials to the corresponding geometric domain indices.
61         Defines the Dirichlet boundary conditions for the BEM problem.
62         """
63
64         #Settings of the potentials
65         pot_apert1 = self.data.pot_apert1
66         pot_apert2 = self.data.pot_apert2
67         pot_electrode13 = self.data.pot_electrode13
68         pot_electrode24 = self.data.pot_electrode24
69         pot_shield = self.data.pot_shield
70
71         apert1=self.data.group_id[0]
72         apert2=self.data.group_id[1]
73         elec1=self.data.group_id[2]
74         elec2=self.data.group_id[3]
75         elec3=self.data.group_id[4]
76         elec4=self.data.group_id[5]
77         shield=self.data.group_id[6]
78
79         @bempp.real_callable
80         #Setting of the potential on the different elements of the geometry
81         #electrode 1 and 2 are coupled and 3 and 4 are coupled
82         def dirichlet_data(x, n, domain_index, result):
83             if domain_index == apert1: #potential of the 1st aperture
84                 result[0]=pot_apert1
85
86             elif domain_index == apert2: #potential of the 2nd aperture
87                 result[0]=pot_apert2
88
89             elif domain_index == elec1: #potential of the 1st electrode
90                 result[0]=pot_electrode13
91
92             elif domain_index == elec2: #potential of the 2nd electrode
93                 result[0]=pot_electrode13
94
95             elif domain_index == elec3: #potential of the 3rd electrode
96                 result[0]=pot_electrode24
97
98             elif domain_index == elec4: #potential of the 4th electrode
99                 result[0]=pot_electrode24
100
101             elif domain_index == shield: #potential of the shield
102                 result[0]=pot_shield
103
104         # Create the GridFunction representing the boundary potential
105         self.dirichlet_fun = bempp.GridFunction(self.p1_space, fun=dirichlet_data)
106         # Après avoir créé self.dirichlet_fun
107         v_max = np.max(self.dirichlet_fun.coefficients)
108         v_min = np.min(self.dirichlet_fun.coefficients)
109         print(f"Potentiel Max sur le maillage : {v_max} V")
110         print(f"Potentiel Min sur le maillage : {v_min} V")
111
112     def visualisation(self) -> None:
113         """Visualize the Dirichlet potential mapped onto the 3D geometry."""
114         self.dirichlet_fun.plot()
115
116     def matrix_inversion(self) -> None:

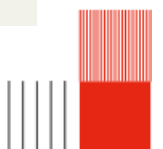
```



```

117 #Sum of the right part of the integral
118 rhs = (-0.5 * self.identity + self.dlp) * self.dirichlet_fun
119
120 #Resolution of the linear system
121 self.neumann_fun, _ = bempp.linalg.cg(self.slp, rhs, tol=1e-8) #1e-5 à tester
122 # mettre function ici -> utiliser resultat de neuman fun
123 #Creation of the tracing of the solution
124 #à changer ici pour verifier
125 n_grid_points = 100
126 self.points = np.stack((np.zeros(n_grid_points), np.zeros(n_grid_points), (np.linspace(0,
10*self.data.total_length, n_grid_points)))) #x=y=0 and z changes from 0 to
elec_id[7][11]=total_length
127
128 #Green's representation
129 slp_pot = bempp.operators.potential.laplace.single_layer(self.dp0_space, self.points) #total matrix
130 self.u_evaluated = -slp_pot * self.neumann_fun
131
132
133 def derivatives(self) -> None:
134     """
135     Calculate the 1st to 4th order derivatives of the potential at the evaluation points.
136     Uses OpenCL for hardware acceleration.
137     """
138     #Field
139     E = bempp.operators.potential.laplace.single_layer_gradient(self.dp0_space, self.points,
device_interface="opencl")
140     self.E_eval = -E * self.neumann_fun
141
142     #Field's 2nd derivative
143     D2 = bempp.operators.potential.laplace.single_layer_2nd_deriv(self.dp0_space, self.points,
device_interface="opencl")
144     self.D2_eval = D2 * self.neumann_fun
145
146     #Field's 3rd derivative
147     D3 = bempp.operators.potential.laplace.single_layer_3rd_deriv(self.dp0_space, self.points,
device_interface="opencl")
148     self.D3_eval = D3 * self.neumann_fun
149
150     #Field's 4th derivative
151     D4 = bempp.operators.potential.laplace.single_layer_4th_deriv(self.dp0_space, self.points,
device_interface="opencl")
152     self.D4_eval = D4 * self.neumann_fun
153
154
155 def potential_exportation(self) -> None:
156     """
157     Export all calculated potentials, derivatives, and geometric parameters
158     to a compressed .npz file for post-processing.
159     """
160     try:
161         from IPython import get_ipython
162         ipython = get_ipython()
163         if ipython is not None:
164             ipython.run_line_magic("matplotlib", "inline")
165             ipython = True
166
167     except NameError:
168         ipython = False
169
170     savefile = os.path.join(self.data.output_dir, self.file_name)
171
172     #Creation of a .npz file with the extracted potential
173     if "savefile" is not None:
174         np.savez_compressed(
175             savefile,
176             #Points
177             points=self.points,
178
179             #Extracted potential
180             potential=self.u_evaluated,
181
182             #Geometry identification
183             group_id_ap1=self.data.group_id[0],

```



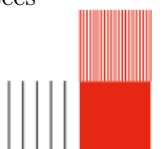
```

184     group_id_ap2=self.data.group_id[1],
185     group_id_cyl1=self.data.group_id[2],
186     group_id_cyl2=self.data.group_id[3],
187     group_id_cyl3=self.data.group_id[4],
188     group_id_cyl4=self.data.group_id[5],
189     group_id_shield=self.data.group_id[6],
190
191     #Mesh parameters
192     MeshSizeMin = self.data.MeshSizeMin,
193     MeshSizeMax = self.data.MeshSizeMax,
194     MeshSizeFromCurvature = self.data.MeshSizeFromCurvature,
195
196     #Output directory
197     output_dir = self.data.output_dir,
198
199     #Dimensions
200     dist_shield_apert = self.data.dist_shield_apert,
201     dist_apert_quad = self.data.dist_apert_quad,
202     radius_ext_shield = self.data.radius_ext_shield,
203     radius_in_shield = self.data.radius_in_shield,
204     thickness_shield = self.data.thickness_shield,
205     radius_apert = self.data.radius_apert,
206     thickness_apert = self.data.thickness_apert,
207     length_cylinder = self.data.length_cylinder,
208     radius_axis = self.data.radius_axis,
209     total_length = self.data.total_length,
210
211     #Coordinates
212     coord_cylinder_x_or_y = self.data.coord_cylinder_x_or_y,
213     coord_cylinder_z = self.data.coord_cylinder_z,
214     coord_apert_z1 = self.data.coord_apert_z1,
215     coord_apert_z2 = self.data.coord_apert_z2,
216     start_shield1 = self.data.start_shield1,
217     end_shield1 = self.data.end_shield1,
218     start_apert1 = self.data.start_apert1,
219     end_apert1 = self.data.end_apert1,
220     start_cyl = self.data.start_cyl,
221     end_cyl = self.data.end_cyl,
222     start_apert2 = self.data.start_apert2,
223     end_apert2 = self.data.end_apert2,
224     start_shield2 = self.data.start_shield2,
225     end_shield2 = self.data.end_shield2,
226
227     #Potentials
228     pot_electrode13 = self.data.pot_electrode13,
229     pot_electrode24 = self.data.pot_electrode24,
230     pot_apert1 = self.data.pot_apert1,
231     pot_apert2 = self.data.pot_apert2,
232     pot_shield = self.data.pot_shield,
233     pot_acceleration = self.data.pot_acceleration,
234
235     #Derivatives
236     E_eval=self.E_eval,
237     D2_eval=self.D2_eval,
238     D3_eval=self.D3_eval,
239     D4_eval=self.D4_eval,)
240
241     print(f"Saved potential to {savefile}.npz")
242
243
244     def potential_axis_printing(self) -> None:
245         """Plot the calculated potential along the Z-axis."""
246         plt.plot(self.points[2], self.u_evaluated[0])
247         plt.title("Potentiel du quadrupole d'Okayama le long de l'axe Z")
248         plt.xlabel("Position en z (mm)")
249         plt.ylabel("Potentiel (V)")
250         plt.show()

```

3.3.4 Extraction data

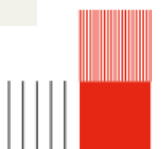
Ce code permet de récupérer toutes les variables créées dans le code Data.py et les données exportées lors de l'extraction du potentiel.



```

1 import numpy as np
2
3 #Class to extract all the necessary informations from the files
4 class Extracted_data:
5     def __init__(self, file_path: str, z_off = None) -> None:
6         """
7         Class to extracte the data of our files
8         Z_off is used only if we use more than one quadrupole
9         """
10
11     # file load
12     data = np.load(file_path)
13
14
15     # axes data
16     self.points = data["points"]
17     self.axe_z = data["points"][2]
18     self.z_off = z_off
19
20     # Potential
21     self.potential = data["potential"]    # Le tableau complet [V]
22
23     # Identifiants de groupes (Physical Groups de GMSH)
24     self.group_id_ap1 = data["group_id_ap1"]
25     self.group_id_ap2 = data["group_id_ap2"]
26     self.group_id_cyl1 = data["group_id_cyl1"]
27     self.group_id_cyl2 = data["group_id_cyl2"]
28     self.group_id_cyl3 = data["group_id_cyl3"]
29     self.group_id_cyl4 = data["group_id_cyl4"]
30     self.group_id_shield = data["group_id_shield"]
31
32     #Mesh parameters
33     self.MeshSizeMin = data["MeshSizeMin"]
34     self.MeshSizeMax = data["MeshSizeMax"]
35     self.MeshSizeFromCurvature = data["MeshSizeFromCurvature"]
36
37     #Output directory
38     self.output_dir = data["output_dir"]
39
40     # Dimensions et géométrie
41     self.dist_shield_apert = data["dist_shield_apert"]
42     self.dist_apert_quad = data["dist_apert_quad"]
43     self.radius_ext_shield = data["radius_ext_shield"]
44     self.radius_in_shield = data["radius_in_shield"]
45     self.thickness_shield = data["thickness_shield"]
46     self.radius_apert = data["radius_apert"]
47     self.thickness_apert = data["thickness_apert"]
48     self.length_cylinder = data["length_cylinder"]
49     self.radius_axis = data["radius_axis"]
50     self.total_length = data["total_length"]
51
52     #Coordinates
53     self.coord_cylinder_x_or_y = data["coord_cylinder_x_or_y"]
54     self.coord_cylinder_z = data["coord_cylinder_z"]
55     self.coord_apert_z1 = data["coord_apert_z1"]
56     self.coord_apert_z2 = data["coord_apert_z2"]
57     self.start_shield1 = data["start_shield1"]
58     self.end_shield1 = data["end_shield1"]
59     self.start_apert1 = data["start_apert1"]
60     self.end_apert1 = data["end_apert1"]
61     self.start_cyl = data["start_cyl"]
62     self.end_cyl = data["end_cyl"]
63     self.start_apert2 = data["start_apert2"]
64     self.end_apert2 = data["end_apert2"]
65     self.start_shield2 = data["start_shield2"]
66     self.end_shield2 = data["end_shield2"]
67
68     # Valeurs des tensions appliquées
69     self.Vacceleration = data["pot_acceleration"]
70     self.Velectrode13 = data["pot_electrode13"]
71     self.Velectrode24 = data["pot_electrode24"]
72     self.Vapert1 = data["pot_apert1"]
73     self.Vapert2 = data["pot_apert2"]

```



```

74     self.Vshield = data["pot_shield"]
75
76     #Derivatives
77     self.D0 = data["potential"][0]      # Le potentiel sur l'axe
78     self.D1 = data["E_eval"]            # Champ électrique [V/mm]
79     self.D2 = data["D2_eval"]           # 2ème dérivée [V/mm^2]
80     self.D3 = data["D3_eval"]           # 3ème dérivée [V/mm^3]
81     self.D4 = data["D4_eval"]           # 4ème dérivée [V/mm^4]
82
83
84     def derivative(self)-> None:
85         """
86         Computation of the derivatives of phi0 to use in paraxial equation
87         """
88         self.D2zphi0 = self.D2[5] #phi_0'' pour la trajectoire
89         self.D1zphi0 = self.D1[2] #phi_0' pour la trajectoire
90
91         #fonction à appeler uniquement si ya plusieurs quad
92         def position_quad(self) :
93             if self.z_off is None:
94                 print("1 seul quad")
95                 return
96
97             self.pos_ap1 = [z + self.coord_apert_z1 for z in self.z_off]
98             self.pos_ap2 = [z + self.coord_apert_z2 for z in self.z_off]
99             self.pos_cyl_start = [z + self.start_cyl for z in self.z_off]
100             self.pos_cyl_end = [z + self.end_cyl for z in self.z_off]

```

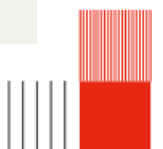
3.3.5 Multipolar decomposition

Ce code permet de calculer les différentes composantes de la décomposition du potentiel.

```

1 import numpy as np
2 from Extraction_data import Extracted_data
3
4 class Decomposition:
5     def __init__(self, data: Extracted_data) -> None:
6         self.data = data
7
8         self.Phi0_maj = None
9         self.Phi1_maj = None
10        self.Phi2_maj = None
11        self.Phi3_maj = None
12        self.Phi4_maj = None
13
14        self.Phi0_fit = None
15        self.Phi2_fit = None
16        self.Phi4_fit = None
17
18
19        def composantes(self) -> None:
20            self.Phi0_maj = -self.data.D0 # potentiel monopolaire sur l'axe
21            self.Phi1_maj = self.data.D1[0] # D1 correspond au champs electrostique -> on met un
22            moins
23            self.Phi2_maj = (1/4)*(self.data.D2[0] - self.data.D2[3])
24            self.Phi3_maj = (1/24) * (self.data.D3[0] - 3*self.data.D3[3])
25            self.Phi4_maj = (1/192)* (self.data.D4[0] + self.data.D4[10] - 6*self.data.D4[3])
26
27            self.Phi0_fit = self.Phi0_maj / self.data.Vapert1
28            self.Phi2_fit = self.Phi2_maj * ((self.data.radius_axis**2)/ self.data.Velectrode13)
29            self.Phi4_fit = self.Phi4_maj * ((self.data.radius_axis**4)/ self.data.Vapert1)
30
31            Phi2_max = np.max(np.abs(self.Phi2_fit))
32
33            self.Phi0_fit = self.Phi0_fit/Phi2_max
34            self.Phi2_fit = self.Phi2_fit/Phi2_max
35            self.Phi4_fit = self.Phi4_fit/Phi2_max

```



Voici un graphique représentant les différentes composantes multipolaires le long de l'axe z :

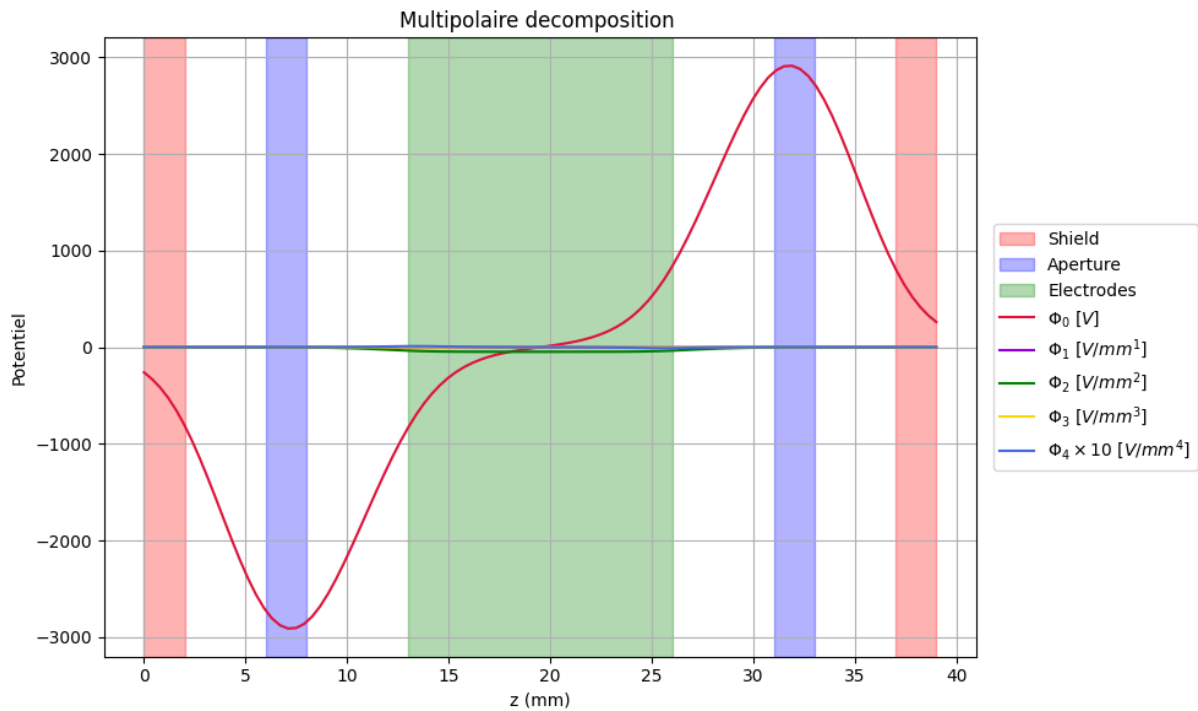


FIGURE 3.3 – Graphique de la décomposition multipolaire

3.3.6 Fit functions

Ce code permet d'implémenter les fonctions de fit présentées dans le papier d'Okayama (Okayama et Kawakatsu, 1978) afin de vérifier si nos codes fonctionnent correctement.

```
1 import numpy as np
2 from Extraction_data import Extracted_data
3
4 #Creation of a class that describes characteristic values for the fit function
5 class Fit_constants:
6     """
7     Defines and calculates the characteristic fitting functions (k0, k2, k4)
8     for multipolar components based on the Okayama model.
9     """
10    def __init__(self, a0: int, b0: int, b2: int, a4: int, b41: int, b42: int, z0: int, a: int,
11                data: Extracted_data) -> None:
12        """
13        Initialize fit parameters for round lens, quadrupole, and octupole components.
14
15        Args:
16            a0, b0: the round lens component (k0).
17            b2: the quadrupole component (k2).
18            a4, b41, b42: the octupole component (k4).
19            z0: Half-length of the plateau for the quadrupole field.
20            a: electrode radius.
21            data (Extracted_data): Object containing axis and geometry information.
22        """
23
24        # Round lens (k0) parameters
25        # k0(Z) = a0 * exp[-(Z/b0)^2]
26        self.a0 = a0
27        self.b0 = b0
28
29        # Quadrupole (k2) parameters
30        self.b2 = b2
```

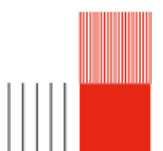


```

29
30     # Octupole (k4) parameters
31     self.a4 = a4
32     self.b41 = b41
33     self.b42 = b42
34     self.z0 = z0
35
36     # Geometry
37     self.a = a
38
39     self.data = data
40
41     # Resulting function arrays
42
43     self.k0 = None
44     self.k2 = None
45     self.k4 = None
46
47
48
49 def fonction_fit(self) -> None:
50     """
51     Calculate the axial distributions for k0, k2, and k4 by shifting the
52     coordinate system to the relevant geometric centers.
53     """
54
55     milieu_cyl = 0.5 * (self.data.start_cyl + self.data.end_cyl)
56
57     # Translate the Z-axis relative to the cylinder center
58     z_ref = self.data.axe_z - milieu_cyl
59
60     # Define reference points for each component
61     z_quad_center = 0 # Center of cylinder
62     z_quad_edge = self.data.length_cylinder / 2 # Edge of cylinder
63     z_ap_center = z_quad_edge + self.data.dist_apert_quad + self.data.thickness_apert / 2
64
65     # Milieu ouverture
66
67     Z_k2 = z_ref - z_quad_center # pour le δquadruple
68     Z_k4 = z_ref - z_quad_edge # pour l'δoctuple
69     Z_k0 = z_ref - z_ap_center # pour la lentille ronde (aperture)
70
71     # k0 (Lentille ronde)
72     a0, b = 0.80751, 5.08
73     self.k0 = a0 * np.exp(-(Z_k0**2) / b**2)
74
75     # k2 (δQuadruple)
76     Z0, b2 = 5, 2.54
77     self.k2 = np.where(np.abs(Z_k2) <= Z0, 1, np.exp(-(np.abs(Z_k2) - Z0)**2 / b2**2))
78
79     # k4 (δOctuple)
80     a4, b1, b2_k4 = 0.03891461, 3.113, 2.015
81     self.k4 = np.where(Z_k4 <= 0, a4 / (1 + (Z_k4**2 / b1**2))**2, a4 * np.exp(-(Z_k4**2) / b2_k4**2))

```

Voici un graphique représentant les différentes composantes multipolaires normées ainsi que les fonctions de fit d'Okayam, le long de l'axe z :



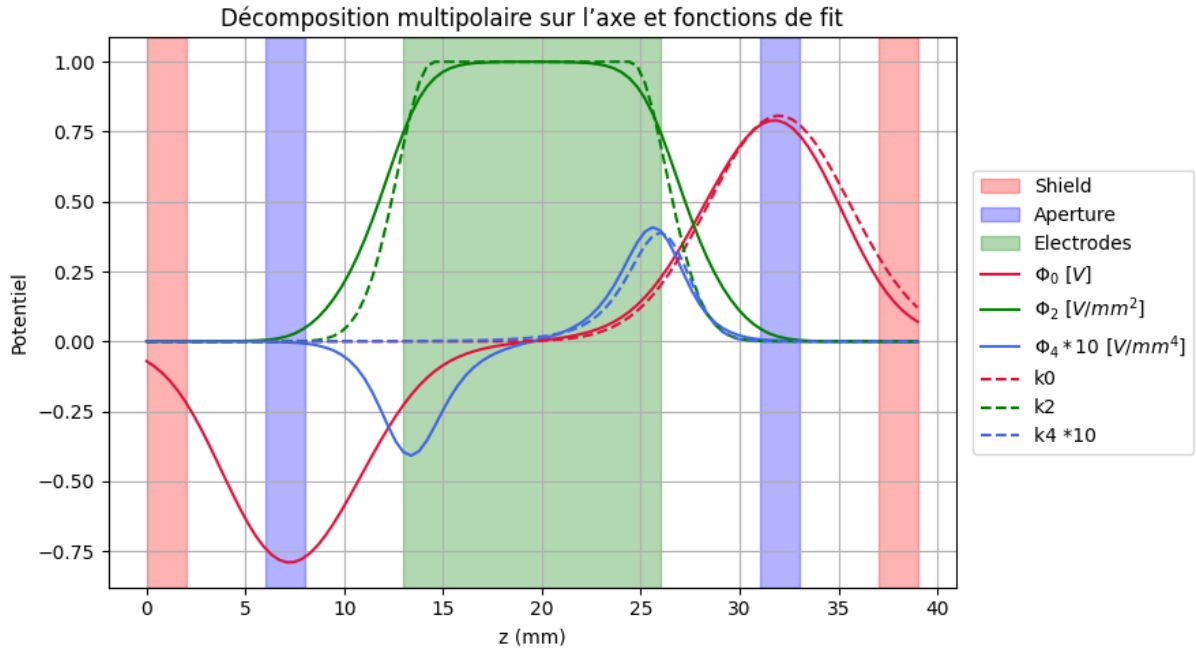


FIGURE 3.4 – Graphique de la décomposition multipolaire et des fonctions de fit

3.3.7 Graphs

Ce code permet d'afficher différents graphiques tels que la valeur des différentes composantes du potentiels le long de l'axe z.

```

1 from matplotlib import pylab as plt
2 import matplotlib.pyplot
3 from Extraction_data import Extracted_data
4 from Multipolar_decomposition import Decomposition
5 from Fit_functions import Fit_constants
6
7 class Graphs:
8     """
9     A class to handle the visualization of multipolar decomposition data
10    and the quadrupole geometry.
11    """
12    def __init__(self, data: Extracted_data, decomposition: Decomposition, fit : Fit_constants
13    = None) -> None:
14        """
15        data (Extracted_data): Object containing axis coordinates and geometry boundaries.
16        decomposition (Decomposition): Object containing multipolar component values.
17        fit (Fit_constants): Object containing Okayama fitting function constants.
18        """
19        self.data = data
20        self.decomposition = decomposition
21        self.fit = fit
22
23        self.ax = None
24
25
26    def trace_geo(self, ax) -> None:
27        """
28        Overlay the physical geometry of the system.
29        """
30        if self.data.z_off is not None :
31            offset = self.data.z_off
32            print(offset)

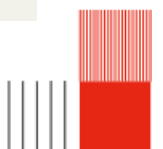
```



```

33
34     # On utilise un drapeau pour ne mettre les labels qu'une seule fois
35     for i, z in enumerate(offset):
36         label_s = 'Shield' if i == 0 else ""
37         label_a = 'Aperture' if i == 0 else ""
38         label_e = 'Electrodes' if i == 0 else ""
39
40         ax.axvspan(z + self.data.start_shield1, z + self.data.end_shield1,
41 color='red', alpha=0.15, label=label_s)
42         ax.axvspan(z + self.data.start_apert1, z + self.data.end_apert1,
43 color='blue', alpha=0.15, label=label_a)
44         ax.axvspan(z + self.data.start_cyl, z + self.data.end_cyl, color='green',
45 alpha=0.15, label=label_e)
46         ax.axvspan(z + self.data.start_apert2, z + self.data.end_apert2,
47 color='blue', alpha=0.15)
48         ax.axvspan(z + self.data.start_shield2, z + self.data.end_shield2,
49 color='red', alpha=0.15)
50     else :
51         # Cas d'un seul quadrupole
52         ax.axvspan(self.data.start_shield1, self.data.end_shield1, color='red',
53 alpha=0.15, label='Shield')
54         ax.axvspan(self.data.start_apert1, self.data.end_apert1, color='blue',
55 alpha=0.15, label='Aperture')
56         ax.axvspan(self.data.start_cyl, self.data.end_cyl, color='green', alpha=0.15,
57 label='Electrodes')
58         ax.axvspan(self.data.start_apert2, self.data.end_apert2, color='blue',
59 alpha=0.15)
60         ax.axvspan(self.data.start_shield2, self.data.end_shield2, color='red',
61 alpha=0.15)
62
63 def graphe_composantes(self) -> None:
64     """
65     Plot the multipolar components (Phi0 to Phi4) along the Z-axis
66     overlaid with the system geometry.
67     """
68     # 1. Initialize figure and axis
69     fig, ax = plt.subplots(figsize=(9, 5))
70
71     # 2. Trace the background geometry
72     self.trace_geo(ax)
73     # Plot Calculated decomposition data and Okayam'a model
74
75     ax.plot(self.data.axe_z, self.decomposition.Phi0_maj, label=r'$\Phi_0$ [V]',
76 color='crimson')
77     ax.plot(self.data.axe_z, self.decomposition.Phi1_maj, label=r'$\Phi_1$ [V/mm]',
78 color='darkviolet')
79     ax.plot(self.data.axe_z, self.decomposition.Phi2_maj, label=r'$\Phi_2$ [V/mm^2]',
80 color='green')
81     ax.plot(self.data.axe_z, self.decomposition.Phi3_maj, label=r'$\Phi_3$ [V/mm^3]',
82 color='gold')
83     ax.plot(self.data.axe_z, 10*self.decomposition.Phi4_maj, label=r'$\Phi_4 \times 10$ [V/mm^4]', color='royalblue')
84
85     # 4. label and style
86     ax.set_xlabel("z (mm)")
87     ax.set_ylabel("Potentiel")
88     ax.set_title("Multipolaire decomposition ")
89     ax.grid(True)
90
91     ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), borderaxespad=0.)
92
93     plt.tight_layout()
94     plt.show()

```



```

81
82 #same but with fitting potential component
83 def graphe_fit(self, bool_fit = True ) -> None:
84     fig, ax = plt.subplots(figsize=(9, 5))
85     self.trace_geo(ax)
86
87     #Printing of the different composants of the potential
88
89     ax.plot(self.data.axe_z, self.decomposition.Phi0_fit, label=r'$\Phi_0$ $[V]$', 
90     color='crimson')
91     ax.plot(self.data.axe_z, self.decomposition.Phi2_fit, label=r'$\Phi_2$ $[V/mm^2]$', 
92     color='green')
93     plt.plot(self.data.axe_z, 10*self.decomposition.Phi4_fit, label=r'$\Phi_4$ *10$ 
94     $[V/mm^4]$', color='royalblue')
95
96     if (bool_fit == True ):
97         ax.plot(self.data.axe_z, self.fit.k0, label=r'k0', color='crimson', 
98         linestyle='dashed')
99         ax.plot(self.data.axe_z, self.fit.k2, label=r'k2', color='green', 
100         linestyle='dashed')
101         ax.plot(self.data.axe_z, 10*self.fit.k4, label=r'k4 *10', color='royalblue', 
102         linestyle='dashed')
103
104     ax.set_xlabel("z (mm)")
105     ax.set_ylabel("Potentiel")
106     ax.set_title("Décomposition multipolaire sur 'laxe et fonctions de fit")
107     ax.grid()
108     ax.legend()
109     ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), borderaxespad=0.)
110     plt.tight_layout()
111     plt.show()

```

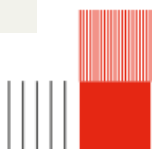
3.3.8 Paraxial

Ce code permet de résoudre les équations de la trajectoire et donc d'obtenir le comportement des rayons d'électrons à travers le quadrupole.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from Data import Data
4 from Extraction_data import Extracted_data
5 from Multipolar_decomposition import Decomposition
6 from scipy.interpolate import interp1d
7
8 class Paraxial():
9     def __init__(self):
10         # On passe les arguments nécessaires au constructeur de Quadrupole
11         self.y_next = None
12
13     def RK4_step(self, f, y, t, h, alpha, beta):
14         f1 = f(y, t, alpha, beta)
15         f2 = f(y + h*f1/2, t + h/2, alpha, beta)
16         f3 = f(y + h*f2/2, t + h/2, alpha, beta)
17         f4 = f(y + h*f3, t + h, alpha, beta)
18         return y + (h/6)*(f1 + 2*f2 + 2*f3 + f4)
19
20 class Ion(Paraxial):
21     def __init__(self, mass, charge, name, x, vx, y, vy):
22         # Utilisation correcte du super() pour remonter la chaîne d'héritage
23         super().__init__()
24
25         self.mass = mass
26         self.charge = charge

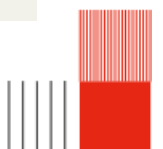
```



```

27     self.name = name
28
29     self.state_x = np.array([x, vx], dtype=float)
30     self.state_y = np.array([y, vy], dtype=float)
31
32     self.history_x = []
33     self.history_y = []
34
35     def afficher_detail(self):
36         print(f"Ion: {self.name}, Masse: {self.mass}, Charge: {self.charge}")
37
38     def save_step(self):
39         self.history_x.append(self.state_x[0])
40         self.history_y.append(self.state_y[0])
41
42 class Trajectoire(Paraxial):
43     def __init__(self):
44         super().__init__()
45         pass
46
47     def equation(self, y, t, alpha, beta) -> None:
48         """
49         forme équation second èdegrs à érsoudre
50         y, t, alpha (coefficient ordre 1), beta écoefficient ordre 2
51         """
52         u = y[0]
53         v = y[1]
54         du = v
55         dv = -alpha * u - beta * v
56         return np.array([du, dv])
57
58     def simulation3(self, ion : Ion, data : Extracted_data, decomp : Decomposition )-> None:
59         """
60         class ion
61         class Data
62         """
63         V_acc = data.Vacceleration
64         print(V_acc)
65
66         dz_mm = data.axe_z[1] - data.axe_z[0]
67         ion.save_step()
68
69         for i in range(len(data.axe_z) - 1):
70             phi_total = V_acc + decomp.Phi0_maj[i]
71
72             if abs(phi_total) < 0.1:
73                 phi_total = 0.1 if phi_total >= 0 else -0.1
74
75             terme_axial = data.D2zphi0[i] / (4 * phi_total)
76             terme_quad = decomp.Phi2_maj[i] / phi_total
77
78             alphax = terme_axial - terme_quad
79             alphay = terme_axial + terme_quad
80             beta = data.D1zphi0[i] / (2 * phi_total)
81
82             #on envoie a RK4 pour érsoudre equa diff
83             ion.state_x = self.RK4_step(self.equation, ion.state_x, data.axe_z[i], dz_mm,
84             alphax, beta)
85             ion.state_y = self.RK4_step(self.equation, ion.state_y, data.axe_z[i], dz_mm,
86             alphay, beta)
87
88             # on sauve la posistion
89             ion.save_step()

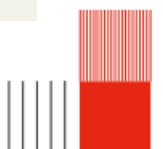
```



```

88
89 def convergence(self, data : Extracted_data, decomp : Decomposition, n : int) -> None:
90     """
91     Data (type extraction)
92     n (entier) ordre convergence que l'on veut évrifier (diminue le pas de l'axe)
93     """
94
95     #on redefinie notre axe z et le pas de l'éintgral
96     n_points= len(data.axe_z)*n
97     self.z_conv = np.linspace(data.axe_z[0], data.axe_z[-1], n_points)
98     self.dz_conv = self.z_conv[1]-self.z_conv[0]
99
100
101
102     # écration de fonctions continues comme ca on peut érduire le pas
103     self.f_phi0 = interp1d(data.axe_z, decomp.Phi0_maj, kind = 'cubic')
104     self.f_phi2 = interp1d(data.axe_z, decomp.Phi2_maj, kind = 'cubic')
105     self.f_phi4 = interp1d(data.axe_z, decomp.Phi4_maj, kind = 'cubic')
106     self.f_D1zphi0 = interp1d(data.axe_z, data.D1zphi0, kind = 'cubic')
107     self.f_D2zphi0 = interp1d(data.axe_z, data.D2zphi0, kind = 'cubic')
108
109
110     #pareil qu'avant avec nos fonctions continues
111     def simulationf(self, ion : Ion, data : Extracted_data ) -> None:
112         """
113         class ion
114         class extraction
115         """
116         V_acc = data.Vacceleration
117         ion.save_step()
118
119         for i in self.z_conv[:-1]:
120             phi_total = V_acc + self.f_phi0(i)
121
122             if abs(phi_total) < 0.1:
123                 phi_total = 0.1 if phi_total >= 0 else -0.1
124
125             terme_axial = self.f_D2zphi0(i) / (4 * phi_total)
126             terme_quad = self.f_phi2(i) / phi_total
127
128             alphax = terme_axial - terme_quad
129             alphay = terme_axial + terme_quad
130             beta = self.f_D1zphi0(i) / (2 * phi_total)
131
132             #envoi a RK4
133             ion.state_x = self.RK4_step(self.equation, ion.state_x, i, self.dz_conv, alphax,
134             beta)
135             ion.state_y = self.RK4_step(self.equation, ion.state_y, i, self.dz_conv, alphay,
136             beta)
137
138             ion.save_step()
139
140
141     def solution_analytique2(self, ion : Ion, data : Extracted_data, decomp : Decomposition, z0
142     = 0) -> None :
143         z_axes = data.axe_z
144         n = len(z_axes)
145
146         # Initialisation des tableaux
147         self.xp = np.zeros(n)
148         self.xm = np.zeros(n)
149         self.yp = np.zeros(n)
150         self.ym = np.zeros(n)

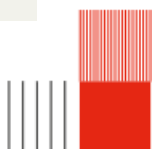
```



```

148     phi0 = abs(data.Vacceleration)
149     print(phi0)
150
151
152     for i in range(n):
153         dist = z_axes[i] - z0
154
155         val_phi2 = abs(decomp.Phi2_maj[i])
156         print
157
158         if val_phi2 < 1e-10:
159             # Cas sans champ (k=0) : mouvement rectiligne
160             self.xp[i] = 1.0
161             self.xm[i] = dist
162             self.yp[i] = 1.0
163             self.ym[i] = dist
164         else:
165             k = np.sqrt(val_phi2 / phi0)
166
167             self.xp[i] = np.cos(k * dist)
168             self.xm[i] = (1/k) * np.sin(k * dist)
169
170             self.yp[i] = np.cosh(k * dist)
171             self.ym[i] = (1/k) * np.sinh(k * dist)
172
173     def solution_analytique(ion,potentiel,quad1, z0): #champ quadrupolaire uniquement
174
175         xp = np.zeros(len(potentiel.axe_z)-1)
176         xm = np.zeros(len(potentiel.axe_z)-1)
177         yp = np.zeros(len(potentiel.axe_z)-1)
178         ym = np.zeros(len(potentiel.axe_z)-1)
179
180         for i in range(len(potentiel.axe_z)-1):
181             z = potentiel.axe_z[i]
182             w0 = np.sqrt(2*ion.charge*potentiel.VQ/ion.masse*(Dimension.radius)**2)
183             xp[i] = np.cos(w0*(z-z0))
184             xm[i] = (1/w0)*np.sin(w0*(z-z0))
185             yp[i] = np.cosh(w0*(z-z0))
186             ym[i] = (1/w0)*np.sinh(w0*(z-z0))
187         return xp,xm, ym, yp
188
189
190
191     #xp,xm, ym, yp = solution_analytique(Ion,data, 0)
192
193     def solution_analytique2(self, ion : Ion, data : Extracted_data, decomp : Decomposition, z0
194     = 0) -> None :
195         #pour un champs quad unique -> on met les apertures a 0
196         z_axes = data.axe_z
197         n = len(z_axes)
198         self.xp, self.xm = np.zeros(n), np.zeros(n)
199         self.yp, self.ym = np.zeros(n), np.zeros(n)
200
201
202         for i in range(len(data.axe_z)):
203             z = data.axe_z[i]
204             vz = data.Vacceleration + decomp.Phi0_maj[i]
205             phi2 = decomp.Phi2_maj[i]
206             w0 = np.sqrt(phi2*ion.charge/ion.mass*(vz)**2)
207             self.xp[i] = np.cos(w0*(z-z0))
208             self.xm[i] = (1/w0)*np.sin(w0*(z-z0))
209             self.yp[i] = np.cosh(w0*(z-z0))

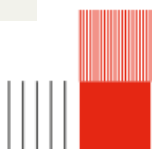
```



```

210         self.ym[i] = (1/w0)*np.sinh(w0*(z-z0))
211         print(self.xp)
212
213     def plot_theorique(self, data : Extracted_data, z0 = 0) -> None :
214         fig, axs = plt.subplots(2, 2, figsize=(15, 10))
215         fig.suptitle("Trajectoire theorique d'un quad seul ", fontsize=14, fontweight='bold')
216
217         ax = axs.flatten()
218
219         # 1. Trajectoire X (Principal vs Marginal)
220         ax[0].plot(data.axe_z, self.xp, 'r-', label="Principal (x)")
221         ax[0].plot(data.axe_z, self.xm, 'b-', label="Marginal (x)")
222         ax[0].set_title("X Trajectories ")
223         ax[0].legend()
224
225         # 2. Trajectoire Y (Principal vs Marginal)
226         ax[1].plot(data.axe_z, self.yp, 'r-', label="Principal (y)")
227         ax[1].plot(data.axe_z, self.ym, 'b-', label="Marginal (y)")
228         ax[1].set_title("Y Trajectories")
229         ax[1].legend()
230
231         # 3. Chief Ray seul (X vs Y)
232         ax[2].plot(data.axe_z, self.xp, 'r-', label=" rayon principal ")
233         ax[2].plot(data.axe_z, self.yp, 'b-', label=" rayon principal ")
234         ax[2].set_title("rayon principal ")
235         ax[2].legend()
236
237         # 4. Marginal Ray seul (X vs Y)
238         ax[3].plot(data.axe_z, self.xm, 'r-', label="rayon marginal")
239         ax[3].plot(data.axe_z, self.ym, 'b-', label="rayon marginal")
240         ax[3].set_title("rayon marginal")
241         ax[3].legend()
242
243
244
245
246         plt.tight_layout(rect=[0, 0.03, 1, 0.95])
247         plt.show()
248
249
250
251
252
253
254
255
256
257
258
259     #plot de la trajectoire avec les fonctions edfinit sur N points
260     def plot_discret(self, principal: Ion, marginal: Ion, data : Extracted_data)-> None:
261
262         fig, axs = plt.subplots(2, 2, figsize=(15, 10))
263         fig.suptitle("Trajectoire paraxiale", fontsize=14, fontweight='bold')
264
265         ax = axs.flatten()
266
267         # 1. Trajectoire X (Principal vs Marginal)
268         ax[0].plot(data.axe_z, principal.history_x, 'r-', label="Principal (x)")
269         ax[0].plot(data.axe_z, marginal.history_x, 'b-', label="Marginal (x)")
270         ax[0].set_title("X Trajectories ")
271         ax[0].legend()
272

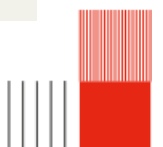
```



```

273 # 2. Trajectoire Y (Principal vs Marginal)
274 ax[1].plot(data.axe_z, principal.history_y, 'r-', label="Principal (y)")
275 ax[1].plot(data.axe_z, marginal.history_y, 'b-', label="Marginal (y)")
276 ax[1].set_title("Y Trajectories")
277 ax[1].legend()
278
279 # 3. Chief Ray seul (X vs Y)
280 ax[2].plot(data.axe_z, principal.history_x, 'r-', label=" rayon principal ")
281 ax[2].plot(data.axe_z, principal.history_y, 'b-', label=" rayon principal ")
282 ax[2].set_title("rayon principal ")
283 ax[2].legend()
284
285 # 4. Marginal Ray seul (X vs Y)
286 ax[3].plot(data.axe_z, marginal.history_x, 'r-', label="rayon marginal")
287 ax[3].plot(data.axe_z, marginal.history_y, 'b-', label="rayon marginal")
288 ax[3].set_title("rayon marginal")
289 ax[3].legend()
290
291
292
293
294 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
295 plt.show()
296
297 plt.figure(2)
298 plt.plot(data.axe_z, marginal.history_x, 'r-', label="rayon marginal")
299 plt.plot(data.axe_z, marginal.history_y, 'b-', label="rayon marginal")
300 plt.xlabel('mm')
301 plt.show()
302
303
304 # plot pour la convergence
305 def plot_continu(self, principal: Ion, marginal: Ion, data : Extracted_data, n) -> None:
306     fig, axs = plt.subplots(2, 2, figsize=(15, 10))
307     fig.suptitle(f"Trajectoire paraxiale, convergence ordre {n} ", fontsize=14,
308 fontweight='bold')
309
310     ax = axs.flatten()
311
312     # 1. Trajectoire X (Principal vs Marginal)
313     ax[0].plot(self.z_conv, principal.history_x, 'r-', label="Principal (x)")
314     ax[0].plot(self.z_conv, marginal.history_x, 'b-', label="Marginal (x)")
315     ax[0].set_title("X Trajectories")
316     ax[0].legend()
317
318     # 2. Trajectoire Y (Principal vs Marginal)
319     ax[1].plot(self.z_conv, principal.history_y, 'r-', label="Principal (y)")
320     ax[1].plot(self.z_conv, marginal.history_y, 'b-', label="Marginal (y)")
321     ax[1].set_title("Y Trajectories")
322     ax[1].legend()
323
324     # 3. Chief Ray seul (X vs Y)
325     ax[2].plot(self.z_conv, principal.history_x, 'r-', label="Principal (x)")
326     ax[2].plot(self.z_conv, principal.history_y, 'r-', label="Principal (y)")
327     ax[2].set_title("Chief Ray ")
328     ax[2].legend()
329
330     # 4. Marginal Ray seul (X vs Y)
331     ax[3].plot(self.z_conv, marginal.history_x, 'b-', label="Marginal (x)")
332     ax[3].plot(self.z_conv, marginal.history_y, 'b-', label="Marginal (y)")
333     ax[3].set_title("Marginal Ray")
334     ax[3].legend()

```




```

335
336
337     plt.tight_layout(rect=[0, 0.03, 1, 0.95])
338     plt.show()
339
340 def plot_faisceau(self , Liste_ion,  data : Extracted_data, ):
341     plt.figure(1)
342     plt.title("Faisceau d'ion")
343     for element in Liste_ion :
344         plt.plot(data.axe_z, element.history_x,label=f"x0 = {element.history_x[0]:.2f} mm" )
345         plt.legend()
346     plt.x_axis = ("z en mm")
347     plt.y_axis = ("axe x en mm")
348     plt.show()

```

